

From Perception to MARL to Navigation: An Integrated Multi-Robot Pursuit System for Factory Environments

Group 2

Mingxuan Song, Morgan Zhang, Yizhong Yan

May 2026

Abstract—This report describes our attempt to build a multi-robot pursuit system with two Unitree Go 2-style agents in a factory-like Isaac Sim / Isaac Lab environment. The project combines a simulation layer, perception/state extraction, a ROS2 Core layer, a MARL decision module, A* and NavDP-style navigation adapters, locomotion adapters, and a dashboard used during integration. The MARL policy acts as the high-level decision module: it receives robot and intruder state estimates and proposes cooperative subgoals, while navigation and locomotion try to execute those subgoals in simulation. Under strict v13 physical constraints, control reaches 88% capture against a stationary intruder, 42% against a random-walk intruder, and 15% against an evasive intruder. These results show both progress and an important limitation: the learned policy can support cooperative pursuit in simplified settings, but the current $(\Delta x, \Delta y)$ action interface does not handle final heading control well enough for robust moving-target capture.

Index Terms—Multi-Agent Reinforcement Learning, MAPPO, Cooperative Encirclement, Unitree Go 2, Transformer Policy, Reward Shaping, A* Pathfinding, NavDP, LiDAR, Sim-to-Real.

I. INTRODUCTION

A. Project goal

This project started from a simple question: can two robot dogs in a factory-like simulator cooperate well enough to trap an intruder, rather than just chase it. The final codebase is bigger than the original MARL experiment. It includes an Isaac Sim / Isaac Lab scene, ROS2 communication, a Core state mirror, perception/state extraction, a MAPPO decision module, navigation adapters, locomotion adapters, a dashboard, and evaluation scripts.

In the intended runtime, simulated sensors or state extraction provide robot and intruder state, MARL proposes one cooperative subgoal per robot, navigation turns those subgoals into waypoints, and locomotion sends commands back to the simulated Go 2-style robots. In practice, different parts of this pipeline matured at different speeds. The MARL environment and reward design were developed most deeply, while perception, NavDP integration, and low-level execution were kept modular so they could be tested and replaced independently during integration.

B. What made the project difficult

A pursuit failure was often hard to diagnose because the final symptom looked the same: the intruder was not captured.

During debugging, however, that same failure could come from quite different sources:

- **State input.** The policy is only as good as the robot and intruder state it receives. In several experiments we used simulator-provided state to keep the rest of the loop testable before a full vision pipeline was ready.
- **Module timing.** Core, planning, locomotion, and simulation do not all run at the same rate. A waypoint that is correct when it is produced can already be stale by the time the robot reaches it, especially when the intruder moves.
- **Cooperation.** The two agents receive a shared objective, but without careful reward shaping they often learn a same-side chase instead of an encirclement.
- **Geometry.** MARL outputs target offsets, not executable paths. The route to a useful encirclement position still has to respect walls, shelves, and short-horizon replanning.
- **Robot heading.** The stricter v13 setup exposed a mismatch in our interface: the policy chose $(\Delta x, \Delta y)$ subgoals, but the final capture condition also depended on yaw and sustained facing.
- **Evidence.** Some system claims need artifacts, not just architecture diagrams: topic freshness, dashboard screenshots, example planner outputs, and videos of the closed loop running.

C. Contributions

The main work completed in this project is:

- a ROS2-based integration structure connecting simulation, perception/state extraction, MARL decision making, navigation, locomotion, dashboard, and evaluation modules;
- an Isaac Sim / Isaac Lab factory environment with robot-mounted cameras, depth, LiDAR, ROS2 publication, and a paired 2D shadow map for efficient MARL training and planning experiments;
- a MAPPO-based cooperative decision module with a Transformer actor, 8-ray LiDAR observation, and thirteen iterations of reward and interface design;
- a modular execution stack connecting MARL subgoal generation to A*, NavDP-style planning adapters, locomotion adapters, and Isaac Sim command execution;

- a Core state mirror and browser dashboard used to inspect robot state, sensor freshness, planning output, locomotion output, and fallback status during integration;
- evaluation under stationary, random-walk, and evasive intruder behaviours;
- a clear failure case showing that MARL’s $(\Delta x, \Delta y)$ subgoal action space is not enough once capture requires heading control.

D. Report structure

Section II reviews related work. Section III describes the integrated system architecture and module interfaces. Section IV explains the Isaac Sim / Isaac Lab simulation environment and 2D shadow map. Section V describes perception and state estimation at the interface level. Section VI presents the MARL decision-making module. Section VII covers navigation and locomotion execution. Section VIII describes runtime integration. Section IX reports experiments and results. Sections X and XI discuss limitations and conclude the report.

II. RELATED WORK

A. PPO and MAPPO

Our system is built on PPO [1], a policy gradient method that clips the surrogate objective to prevent too-large policy updates. It is not the most sample-efficient algorithm, but it is stable and easy to tune, which mattered for a project that went through thirteen reward versions. Yu et al. [2] showed that PPO with a centralised critic, MAPPO, works surprisingly well on cooperative benchmarks like SMAC, even compared to more complex off-policy methods like QMIX [3] or MADDPG [4]. The key idea we take from MAPPO is the centralised critic: during training, the critic sees both agents’ observations, even though each agent’s policy only uses its own observation at test time. We referenced the official MAPPO codebase [5] when setting up our trainer.

QMIX [3] and MADDPG [4] are reasonable alternatives, but we did not use them. QMIX requires value decomposition that is hard to justify when both agents share a single reward, and MADDPG’s off-policy nature made it harder to reproduce stable training runs across reward re-designs. MAPPO’s simplicity was the right trade-off for iterative development.

B. Transformers in RL

We switched from an MLP to a Transformer actor in v10. The foundational architecture is from Vaswani et al. [6]. Decision Transformer [7] framed offline RL as sequence modelling, showing that a GPT-style model conditioned on return-to-go can match or beat TD learning on offline benchmarks. Our use is different: we train online with PPO, and the sequence model serves as a memory rather than a goal-conditioner. The actor takes a 50-frame observation history as input and produces the current action via causal self-attention. The practical reason we made this switch is concrete: MLP agents kept flipping roles every few steps, with both agents converging on the same side of the intruder and then swapping chaotically, while the Transformer agents would commit to one flanking slot and

hold it. Whether this is “true” memory or just a smoothing effect from averaging over history is an open question, but the behavioural improvement was clear.

C. Pursuit-evasion and cooperative encirclement

Pursuit-evasion is a well-studied problem, but most of the existing literature differs from our setup in at least one important way.

Classical control-theoretic approaches. Work like Vidal et al. [8] assumes continuous-space dynamics, often treats the evader as adversarial and optimal, and derives pursuit policies through differential game theory or probabilistic filtering. These methods typically assume agents can communicate or have access to global state, and the environments are open with no obstacles. Our setting has partial observability, obstacles, and non-holonomic heading-locked dynamics, which makes closed-form solutions less feasible.

Learned cooperative pursuit. Chung et al. [9] survey the broader field and note a split between reactive, greedily nearest-agent, and coordinated, role-based pursuit. Several MARL papers train pursuit policies on simple grid worlds or continuous 2D planes without obstacles. The coordination problem there is mostly handled implicitly through shared rewards: agents learn to spread out because clustering is inefficient. In our warehouse environment, the obstacle structure actively disrupts simple spreading strategies. Agents trained without LiDAR, in v1–v2, walked into dead ends and got stuck; the reward gradients from the encirclement objective were too weak to overcome the local optima created by the island geometry.

Encirclement control. There is a body of work on geometric encirclement, where the goal is to surround a target with agents at equal angular spacing [10]. These methods often rely on explicit inter-agent communication to compute each agent’s desired bearing. We do not have communication. Our encirclement condition is a binary check, angle $\geq 90^\circ$ plus facing, not a smooth angular formation, which is easier to reward but harder to satisfy reliably under physical constraints.

What makes our setup different. The combination that we did not find addressed in prior work is:

- 1) no inter-agent communication at execution time;
- 2) heading-locked unicycle-style dynamics with a yaw rate limit;
- 3) a strict temporal capture condition, namely a 2-second hold, rather than an instantaneous proximity check;
- 4) an obstacle-rich indoor environment with partial observability.

The temporal hold requirement in particular is what broke our RL policy in v13: most pursuit-evasion papers define capture as a single-frame distance threshold, so the hold constraint is not something existing reward designs account for. We had to build a geometric fallback controller specifically for this.

D. Reward shaping

Ng et al. [11] proved that potential-based reward shaping leaves the optimal policy unchanged. We used this as a design principle for intermediate reward terms: the progress reward and encirclement angle reward are designed to be monotone functions of position quality, so they should not introduce incorrect equilibria. That said, the proof assumes tabular MDPs or exact function approximation. With a neural network and a 21-dimensional partially observable state, the guarantee is approximate at best. In practice, we still saw reward hacking, including v1 camping and v5–v8 orbiting, which suggests the shaping was not always tight enough to prevent bad local optima.

E. Simulation and sim-to-real

The 3D simulation runs in NVIDIA Isaac Lab [12], built on Isaac Sim [13]. Rudin et al. [14] used a similar setup to train quadruped locomotion in under four minutes using massive parallelisation. Our 2D shadow environment mirrors the same warehouse geometry so that the MARL policy can transfer to the full 3D simulation without significant geometry mismatch. We did not apply domain randomisation [15], which means the current results should be interpreted as simulation-level validation rather than a complete sim-to-real deployment result.

F. NavDP

NavDP (Navigation Diffusion Policy) [16] is an end-to-end mapless navigation framework for robotic navigation tasks. Unlike traditional modular navigation methods that rely on accurate localization, mapping, and manually tuned components, NavDP uses a diffusion-policy-based trajectory generation model to produce candidate navigation trajectories from local RGB-D observations, and then evaluates and selects the most suitable trajectory using a critic function. A key advantage of NavDP is that it leverages privileged information in simulation to generate large-scale, high-quality training data, enabling zero-shot sim-to-real transfer using simulation-only training. Experimental results show that NavDP generalizes well across different robot embodiments, including quadruped robots, wheeled robots, and humanoid robots, and can perform real-time path planning and obstacle avoidance in complex indoor and outdoor environments. Therefore, in this project, NavDP can be used as the low-level path planning module, converting target points or intermediate waypoints provided by the upper-level decision-making module into executable navigation paths.

G. A* pathfinding

The A* algorithm [17] is used as a submodule: the MARL policy outputs a subgoal, and A* finds a collision-free path to it on a 40×40 occupancy grid. This cleanly decouples high-level decision, namely where to go, from low-level navigation, namely how to avoid walls. The main limitation of this design, which we only discovered when testing against a moving intruder, is that A* at 10 Hz is not fast enough to smoothly

track a target that changes direction. The planner reruns every frame, but the latency causes the agent to stutter at waypoints rather than flowing toward the target.

H. Robot perception and tracking

The perception side of the project follows a common robotics pattern: raw sensor streams are converted into compact state estimates before being passed to the decision layer. Modern object detectors such as YOLO [18] are often used for real-time target detection because they produce image-space bounding boxes in a single forward pass. Tracking-by-detection methods such as SORT [19] then associate detections across frames and estimate target motion with lightweight filtering. In a robotics system, this image-space output must still be grounded in the robot frame or world frame using depth, camera calibration, semantic labels, LiDAR, or simulator state.

Our current system uses this separation as an interface principle. MARL does not consume camera images directly; it receives positions, velocities, and target visibility information. This keeps the decision module independent from the perception implementation. In the simulation prototype, ground-truth or simulator-derived state can stand in for the perception estimate so that the ROS2, MARL, navigation, and locomotion loop can be validated before full visual tracking is finalised. This is a pragmatic staging choice rather than a claim that perception is solved.

I. Modular ROS2 robotic systems

ROS2 is designed for modular robot software systems that must connect sensing, planning, control, visualization, and deployment components across process and machine boundaries [20]. This matches our system architecture: Isaac Sim publishes state and sensor topics, Core mirrors the latest state and routes module requests, MARL produces subgoals, NavDP produces waypoints, Locomotion produces executable commands, and the dashboard observes the mirrored state without entering the control loop.

This modular design is useful but introduces interface risk. Each module must agree on coordinate frames, timestamps, topic names, message schemas, update rates, and stale-data handling. In our project, several important failures were not purely learning failures: a valid MARL subgoal could still fail if the planner returned a stale path, if locomotion did not receive the right body-frame command, or if the final heading constraint was handled by the wrong layer. The related systems lesson is therefore that a successful learned policy is only one part of a robotics stack; the module interfaces and runtime arbitration rules are equally important.

III. INTEGRATED SYSTEM ARCHITECTURE

A. Overall pipeline

The system is organised as a perception–decision–navigation loop around an Isaac Sim / Isaac Lab factory environment:

Isaac Sim → Core ROS2 → Perception → MARL → Navigation
→ Locomotion → Isaac Sim

The main design goal is modularity. All modules can be swapped out and tested individually. This will greatly facilitate our future development and modifications, preventing algorithm conflicts. It also allows us to select the appropriate modules to complete tasks in different environments and scenarios. The system does not need to spend a significant amount of time on frequent end-to-end testing; instead, each module can be tested separately. For example, MARL can be tested and trained in a standalone 2D environment.

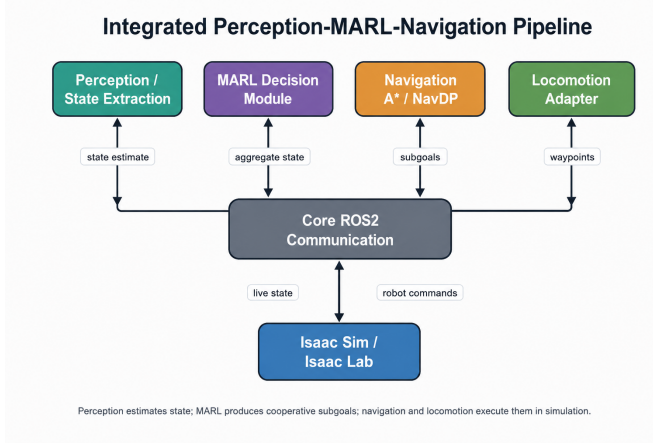


Fig. 1. Full perception–decision–navigation pipeline. Perception/state extraction produces compact robot and intruder state estimates, the Core ROS2 layer routes aggregate state, MARL outputs cooperative subgoals, and navigation plus locomotion executes those subgoals in Isaac Sim / Isaac Lab.

B. Runtime modules

Simulation module. Isaac Sim / Isaac Lab owns the 3D factory environment, robot execution, intruder motion, and simulated sensor streams.

Perception and state extraction. This module converts simulated sensor or ground-truth state into compact robot and intruder estimates: position, velocity, heading when available, and target-detection status.

Core ROS2 communication layer. The core layer aggregates state, synchronises messages, exposes a dashboard state API, and routes requests between perception, MARL, navigation, and locomotion.

MARL decision module. The MARL module receives aggregate state and outputs one high-level subgoal for each robot. It handles role allocation, cooperative encirclement, and strategic pursuit behaviour.

Navigation module. Navigation receives MARL subgoals and turns them into collision-free waypoint paths using A*, NavDP, Nav2, or a waypoint-router implementation.

Locomotion adapter. The locomotion layer converts waypoint or velocity commands into commands that the simulated Go 2-style robots can execute.

Dashboard and logging. The dashboard is read-only. It visualises live robot state, target state, subgoals, fallback mode, and health status during debugging and demonstrations.

C. Communication interfaces

Table I summarises the conceptual interfaces. The exact topic and service names may differ between integration versions, but the content and ownership boundaries should remain stable.

TABLE I
CONCEPTUAL MODULE INTERFACES

Interface	Publisher	Subscriber	Message content
Sensor state	Isaac	Core	Camera image, lidar, imu
Perception state	Perception	Core	Robot poses, intruder pose, velocities
Decision request	Core	MARL	Aggregate state and map context
Subgoal output	MARL	Core	Robot ID, $(\Delta x, \Delta y)$ or world subgoal
Waypoints	Navigation	Core	Path points or local target
Motion command	Locomotion	Core	<code>cmd_vel</code> or joint target command
Motion command	Core	Isaac	<code>cmd_vel</code> or joint target command
Dashboard state	Core	Dashboard	Latest state, health, fallback mode

D. Execution loop

At runtime, the system repeatedly performs the same high-level loop: Isaac Sim publishes state, perception produces compact estimates, Core aggregates the estimates, MARL generates subgoals, navigation generates waypoints, locomotion sends commands, and Isaac Sim advances the robots. This closed-loop structure is the main difference between the full project and a standalone MARL training environment.

IV. SIMULATION ENVIRONMENT

A. Isaac Sim / Isaac Lab setup

The integrated system is validated in an Isaac Sim / Isaac Lab factory-style environment. The simulator provides the physical scene, robot execution backend, intruder motion, and sensor streams used by the perception and ROS2 integration layers. At this stage, the system should be interpreted as a simulation-first prototype rather than a real-robot deployment.

The main simulation entry point is a standalone Isaac Sim / Isaac Lab runtime that loads the warehouse scene, spawns two robot dogs and one humanoid intruder, attaches sensors to the robot bodies, and publishes the Simulation–Core ROS2 contract. This separation is important for the project structure: simulation owns the physical world and sensor generation, while Core, MARL, NavDP, and Locomotion run as separate services outside the simulator.

B. Factory map and obstacle layout

The main environment is a 20×20 m warehouse-like map with perimeter walls and internal shelf islands. These obstacles are important because the task is not only to chase the intruder, but to intercept it while respecting collision constraints and narrow indoor geometry. Spawn positions for the two robot agents and the intruder are sampled from safe regions so that episodes begin in feasible open space.

The Isaac scene contains a flat ground plane, perimeter walls, interior obstacles, and warehouse-style clutter that creates occlusion and path-planning constraints. The 2D MARL environment mirrors this geometry so that the learned decision module and the 3D execution stack share the same high-level spatial structure. This avoids training a policy on an empty pursuit plane and then testing it in a cluttered simulator.

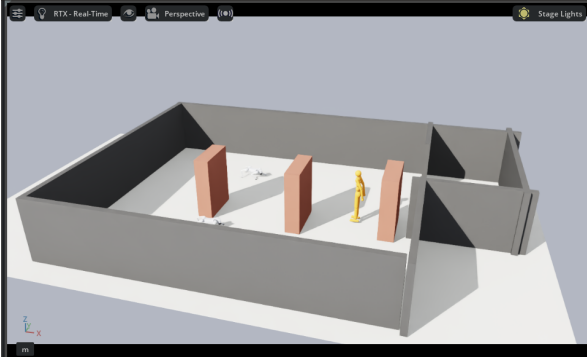


Fig. 2. Isaac Sim / Isaac Lab factory environment used for integrated closed-loop evaluation, showing two Unitree Go 2-style agents, the intruder, and the warehouse obstacle layout.

C. Robot and intruder models

The pursuers are two Unitree Go 2-style quadruped agents. In the MARL training environment, they are represented by simplified kinematic agents so that policy iterations are fast. In the integrated simulation, the navigation and locomotion layers are responsible for turning high-level subgoals into robot motion.

The intruder is represented as a moving target with several evaluation behaviours: stationary, random walk, and evasive. The random-walk and evasive settings are used to test whether the system can handle dynamic pursuit rather than only static encirclement.

In the integrated Isaac Sim runtime, the actors are published under stable IDs, `agent_1`, `agent_2`, and `intruder_1`. The simulation publishes each actor’s pose in the world frame and accepts control commands from Core. The intruder can be held stationary for controlled tests or moved along a configured route for closed-loop pursuit demonstrations.

D. Sensors and ROS2 publication

Each robot is equipped with a front RGB camera, depth output, and an RTX LiDAR stream. The ROS2 bridge publishes robot pose, intruder pose, camera frames, depth images, LiDAR scans, LiDAR point clouds, locomotion observations, and an aggregate debug state under the `/factory/simulation` namespace. The current ROS2-facing contract includes the following relative topics:

- robot and intruder pose topics, using `PoseStamped`;
- front RGB and depth image topics, using `Image`;
- LiDAR scan and point-cloud topics, using `LaserScan` and `PointCloud2`;
- per-robot locomotion observation topics, carrying the 48-D Go 2 observation as JSON;
- an aggregate state topic for debugging and dashboard mirroring.

This gives the rest of the system a realistic sensor-facing interface even when some evaluations use simulator-provided state for repeatability. A mock ROS2 publisher is also included

so that Core and dashboard integration can be tested without launching the full Isaac Sim runtime.

E. 2D shadow environment

Alongside the 3D simulator, we maintain a 2D shadow environment with the same warehouse geometry. This 2D environment is used for fast MARL training, LiDAR-style ray casting, A* path planning, and ablation experiments. Training directly in full 3D Isaac Sim would make reward iteration and policy debugging much slower. The 2D version therefore acts as a controlled decision-making testbed, while the 3D simulator validates integration and execution.

F. Recording and inspection

The simulation runtime supports live inspection from the world camera or robot-mounted camera views. It can also record both a monitor-style view and a top-down view. These recordings were useful during integration because they made it possible to distinguish policy-level failures, path-following failures, and final heading-control failures after an episode had completed.

G. Simulation assumptions and limitations

The current results are simulation-level results. They do not yet include real camera calibration, real quadruped localisation drift, actuator delay, sensor noise, or domain randomisation. These factors matter for sim-to-real transfer and should be treated as future work rather than solved problems in the present system.

V. PERCEPTION AND STATE ESTIMATION

A. Role in the current system

The decision module in this project does not consume raw images. It expects a compact state vector containing the two robot states, the intruder state, and a target-detection indicator. During policy learning this state was provided by the 2D environment or by simulator-side state extraction, which kept the training and integration loop repeatable while the perception stack was developed and validated independently. The perception module described in this section is the implemented sensing path for producing this state schema from raw sensors. In the integrated 3D loop, this lets Core provide estimated state to the same policy interface used during MARL evaluation.

The state passed into MARL is

$$z_t = \{p_t^1, v_t^1, \psi_t^1, p_t^2, v_t^2, \psi_t^2, p_t^I, v_t^I, d_t\}, \quad (1)$$

where p_t^i , v_t^i , and ψ_t^i denote robot i ’s planar position, planar velocity, and yaw, p_t^I and v_t^I denote the intruder position and velocity, and d_t is the detection flag. Yaw is listed explicitly because the implemented self-localization module estimates a full orientation (Section V-D). In the current integrated system, this heading estimate is available to Core and to the capture-condition evaluator. However, the MARL policy used in the reported v13 experiments still outputs only $(\Delta x, \Delta y)$ subgoals and does not directly command final heading. This distinction

helps explain the v13 results discussed later: perception can provide yaw, but the decision/action interface does not yet use heading control as a first-class action.

At a high level, perception runs two estimators on the simulation timeline: robot self-localization, and intruder detection followed by multi-sensor fusion. Fig. 3 summarizes this structure; the remainder of this section details each stage—robot self-localization (Section V-D), intruder detection (Section V-E), and fusion (Section V-F).

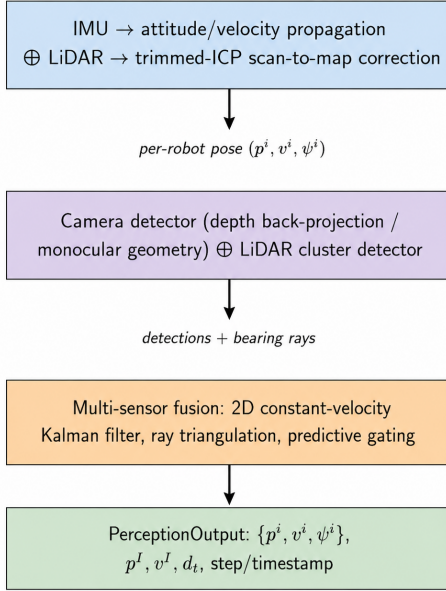


Fig. 3. Perception pipeline. Robot self-localization fuses high-rate IMU propagation with low-rate LiDAR scan matching; intruder localization fuses camera and LiDAR detections through a planar Kalman filter. Both paths emit world-frame estimates on the simulation timeline.

B. Module boundary and runtime contract

Perception is isolated from both the simulator and the decision layer. Isaac Sim publishes camera, depth, LiDAR, semantic, and pose streams; Core mirrors the newest messages; the perception service returns the compact aggregate state. The runtime path is

Isaac Sim sensors → Core mirror → Perception service → Core aggregate state → MARL.

This boundary is enforced by an HTTP service. The simulation/Core process posts a pickled `EnvironmentSensorFrame` (or a `PerceiveRequest` wrapper) to `POST /perceive` and receives a pickled `PerceptionOutput`; `GET /health` is a liveness probe. The pipeline is stateful (it carries Kalman-filter and self-localization state across calls), so requests are serialized by a lock and a single pipeline instance owns its estimator state machine. The request wrapper carries `update_dogs` and `update_intruder` flags so that slim, camera-less frames

can cheaply update dog self-localization without paying for the full intruder fusion pass.

Three contract rules govern the module and are relevant to the estimates it produces:

- 1) **Sensor-only inputs.** Perception does not read the Isaac scene and never consumes the low-level locomotion observation vector; coupling the sensing chain to a policy observation is explicitly rejected. All routing is through Core.
- 2) **World-frame outputs.** All positions, velocities, and orientations follow the Isaac Lab world-frame convention (`root_pos_w`, `root_lin_vel_w`, `root_quat_w`).
- 3) **Simulation time base.** Updates use the simulation timestamp, not wall-clock time. The integration step is recovered dynamically as $\Delta t_k = \tau_k - \tau_{k-1}$, so a low real-time factor does not corrupt velocity estimates or the perception cadence.

Ground truth is mirrored into the frame but is used only for diagnostics (per-step XY position error and yaw error); it never enters the estimators.

C. Sensor inputs available in simulation

Each robot exposes a front RGB camera, a depth image, and a LiDAR stream; the scene additionally carries semantic labels for the robot dogs, the intruder, walls, floor, pillars, and boxes; fixed CCTV-style cameras provide RGB and semantic segmentation but typically no depth. The perception inputs are therefore:

- robot-mounted RGB and depth images for short-range, depth-resolved target localization;
- fixed CCTV RGB plus semantic segmentation for global, bearing-only target localization;
- semantic segmentation, used by the implemented detector to obtain the suspect pixel mask;
- LiDAR point clouds, used both for robot self-localization (scan-to-map) and for dynamic-cluster intruder detection;
- IMU (angular velocity, linear acceleration, orientation/projected gravity) for high-rate self-localization;
- simulator pose data, used only as a diagnostic reference;
- timestamps and step indices for synchronising estimates with MARL decisions on the simulation timeline.

Fig. 4 shows representative examples of these streams.

D. Robot self-localization

Robot self-localization estimates each robot’s planar pose, heading, and planar velocity by combining cheap high-rate IMU dead reckoning with accurate but expensive low-rate LiDAR scan-to-map correction. IMU integration alone drifts without bound; LiDAR matching alone is costly and aliases in symmetric/corridor-like geometry. The estimator therefore propagates with the IMU every step and corrects with LiDAR every N steps ($N = 6$ in the current configuration).

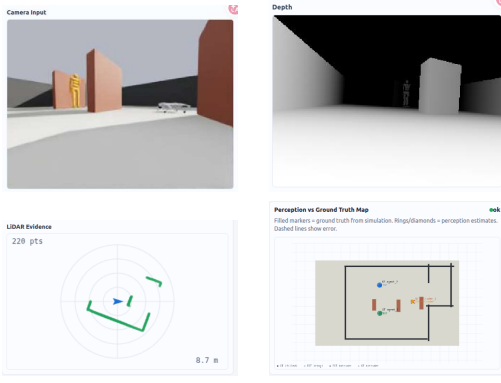


Fig. 4. Example perception inputs available from the Isaac Sim runtime: front RGB image, depth output, LiDAR scan/point cloud, and semantic scene information.

a) IMU propagation.: Let $\omega_b = (p, q, r)$ be the body angular rate and a_b the body specific force. Attitude is propagated with the Euler kinematic relations

$$\dot{\phi} = p + (q \sin \phi + r \cos \phi) \tan \theta, \quad (2)$$

$$\dot{\theta} = q \cos \phi - r \sin \phi, \quad (3)$$

$$\dot{\psi} = \frac{q \sin \phi + r \cos \phi}{\cos \theta} - b_\psi, \quad (4)$$

where b_ψ is an online-estimated gyro yaw bias. Roll and pitch are kept observable by a complementary blend with the gravity direction $g_b = (g_x, g_y, g_z)$ from the IMU,

$$\phi^{\text{grav}} = \text{atan2}(g_y, -g_z), \quad (5)$$

$$\theta^{\text{grav}} = \text{atan2}(-g_x, \sqrt{g_y^2 + g_z^2}), \quad (6)$$

$$\phi \leftarrow (1 - \alpha) \phi^{\text{pred}} + \alpha \phi^{\text{grav}}, \quad (7)$$

$$\theta \leftarrow (1 - \alpha) \theta^{\text{pred}} + \alpha \theta^{\text{grav}}, \quad (8)$$

with a small gain α (so the attitude tracks gravity without absorbing its noise). Planar velocity is obtained by rotating the body specific force into the world frame, removing gravity, and integrating,

$$a^w = R(\hat{q}) a_b - [0, 0, g]^\top, \quad (9)$$

$$v_{xy}^w \leftarrow v_{xy}^w + a_{xy}^w \Delta t, \quad (10)$$

$$p_{xy}^w \leftarrow p_{xy}^w + v_{xy}^w \Delta t, \quad (11)$$

with the vertical channel locked to the initial root height (the estimator is planar). Three mechanisms bound dead-reckoning drift: a zero-velocity update (ZUPT) that collapses the velocity when the robot is detected stationary (small yaw rate, small planar specific force, small estimated speed); an optional fusion of a commanded odometry hint when Core supplies it ($v_{xy}^w \leftarrow 0.15 v_{xy}^w + 0.85 v_{xy}^{\text{odom}}$ and a 0.35/0.65 blend on yaw rate); and velocity damping plus a planar speed clamp, with stronger damping while LiDAR corrections are unavailable.

b) Drift-free yaw reference.: The odometry yaw rate is also integrated into a separate, correction-free reference ψ^{ref} . Each step $\dot{\psi}$ is pulled weakly (5%) toward ψ^{ref} , which prevents slow yaw drift during long LiDAR-outage windows while

remaining weak enough that a reliable scan match can still fully override heading.

c) LiDAR scan-to-map correction.: A dense 2D point map is built once from the static scene geometry by sampling the outlines of all walls, pillars, and boxes at a fixed spacing; no prior mapping run is required. At a correction step, world-frame LiDAR hits are transformed back into the sensor frame, range/height filtered, and voxel downsampled to a bounded scan $S = \{s_k\}$. The sensor-to-robot extrinsic (planar offset and yaw) is self-calibrated from the first scan. Alignment is a trimmed iterated-closest-point (ICP) registration against the static map $M = \{m_j\}$: at each iteration, nearest-map correspondences within a gating distance are formed, the worst correspondences are trimmed (the best $\rho=75\%$ of gated pairs are kept), and the SE(2) update is the closed-form rigid fit

$$(R^*, t^*) = \arg \min_{R \in SO(2), t \in \mathbb{R}^2} \sum_{k \in \mathcal{I}} \|R s_k + t - m_{\pi(k)}\|^2, \quad (12)$$

solved by the SVD of the cross-covariance, with per-iteration translation and yaw deltas clamped to suppress jumps. The match quality combines inlier coverage and residual fit,

$$\text{score} = \underbrace{\frac{|\mathcal{I}|}{|\mathcal{S}|}}_{\text{cov.}} \cdot \underbrace{\frac{1}{1 + \text{rmse}}}_{\text{fit}}, \quad (13)$$

and a correction is accepted only if the inlier count, RMSE, and total correction stay within bounds. On acceptance, position and yaw are reset to the matched pose, velocity is corrected with a quality-weighted blend of the implied LiDAR displacement, and the gyro yaw bias b_ψ in Eq. (4) is nudged from the residual.

d) Robustness.: Three failure modes observed during integration are explicitly handled. (i) *Symmetric initialization*: before the robot is localized, ICP is run from the predicted pose and from a yaw sweep $\{\pm \frac{\pi}{2}, \pi, \pm \frac{\pi}{4}\}$, and a high-inlier low-RMSE candidate can bootstrap the track even below the steady-state score gate. (ii) *Corner aliasing*: a stability gate rejects an otherwise-accepted match whose score falls below an adaptive floor (a fraction of the running accepted-score EMA), whose implied speed or yaw rate is non-physical, or whose yaw disagrees with the drift-free reference by more than a tolerance for a merely mediocre match. Rejected matches fall back to IMU propagation, and a strong-but-rejected match may still contribute a bounded “soft yaw hint”. (iii) *Loss of lock*: repeated rejections clear the localized flag, which re-enables the bootstrap search.

E. Intruder detection

The intruder carries no cooperating sensor, so it is localized purely by being observed. Three detectors run per frame.

a) Depth-resolved camera detection.: The semantic segmentation channel yields the suspect pixel mask (the suspect class ID is resolved once from the segmentation info table and cached per camera). For cameras with valid depth (the robot-mounted cameras), the masked depth is back-projected to the

camera optical frame and transformed to the world frame using the camera pose,

$$P^w = R(q_c) K^{-1} \begin{bmatrix} u \\ v \\ Z \end{bmatrix} + c, \quad (14)$$

followed by a robust 3D outlier rejection (points far from the median are discarded) and a median-based centroid, which is reported with `used_depth = true` and a meaningful world-frame Z .

b) Monocular camera detection.: Fixed CCTV cameras provide no depth, so two complementary geometric cues are combined. A foot point is obtained by intersecting the bottom-centre pixel ray with the ground plane $Z = 0$,

$$r^w = \frac{R(q_c) K^{-1} [u, v, 1]^\top}{\|R(q_c) K^{-1} [u, v, 1]^\top\|}, \quad (15)$$

$$t = \frac{-c_z}{r_z^w}, \quad P_{\text{foot}}^w = c + t r^w, \quad (16)$$

and a body-centre point is obtained from a nominal human height H and the bounding-box pixel height h via $Z \approx f_y H/h$. The reported position interpolates the two by an occlusion factor β ,

$$P^w = (1 - \beta) P_{\text{foot}}^w + \beta P_{\text{body}}^w, \quad (17)$$

where β grows with an occlusion score derived from foot/height depth disagreement and missing lower support. Each monocular detection also exports a world-frame bearing ray (c, r^w) for multi-camera triangulation, and a visibility score (bounding-box size, fill ratio, lower support) and the occlusion score, which act as fusion weights. Implausible projections (off the walkable map, or far from the current track) are rejected.

c) LiDAR cluster detection.: LiDAR localizes the intruder as scene geometry that is not part of the static map: valid hits are height filtered to a person band, points within a radius of known static objects are removed, the remainder is grown into clusters by a greedy distance-based aggregation, and the centroid of the largest cluster is reported. Confidence is the mean of a point-count term and a compactness (spread) term.

F. Multi-sensor fusion

Detections are fused by a planar constant-velocity Kalman filter, chosen over exponential smoothing because the tracker must (i) predict through short target-loss intervals, (ii) output a velocity for the MARL state, and (iii) provide a predicted state for measurement gating. The filter state is $x = [x, y, \dot{x}, \dot{y}]^\top$ with

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (18)$$

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix},$$

and the standard predict/update recursion (Joseph-form covariance update for numerical stability)

$$x_{k|k-1} = F x_{k-1}, \quad (19)$$

$$P_{k|k-1} = F P_{k-1} F^\top + Q, \quad (20)$$

$$y_k = z_k - H x_{k|k-1}, \quad (21)$$

$$S_k = H P_{k|k-1} H^\top + R, \quad (22)$$

$$K_k = P_{k|k-1} H^\top S_k^{-1}, \quad (23)$$

$$x_k = x_{k|k-1} + K_k y_k. \quad (24)$$

a) Measurement construction.: At each step the filter predicts first, and the predicted XY anchors the gating. When two or more monocular bearing rays are available, an intruder XY is recovered by weighted least squares from the ray normals n_i ,

$$\left(\sum_i w_i n_i n_i^\top \right) \hat{x} = \sum_i w_i n_i n_i^\top o_i, \quad (25)$$

with iterative trimming of rays inconsistent with the intersection or pointing backward. Per-camera and per-LiDAR detections are additionally weighted by a soft predictive gate

$$w(\delta) = \begin{cases} 1, & \delta \leq g, \\ \left(1 - \frac{\delta - g}{g}\right)^2, & g < \delta < 2g, \\ 0, & \delta \geq 2g, \end{cases} \quad (26)$$

whose effective radius widens with predicted speed $\|\hat{v}\|$ and detection confidence c and shrinks with occlusion o ,

$$g = (g_0 + \kappa \|\hat{v}\| + \lambda c) (1 - 0.45 o). \quad (27)$$

Each modality is consolidated into a single measurement by a median pre-filter followed by a gated weighted mean. When the camera and LiDAR consensus disagree beyond a threshold, the LiDAR weight is strongly attenuated if the camera consensus is supported (LiDAR is the lower-trust modality here); a LiDAR-only measurement far from the prediction is dropped.

b) Re-acquisition and outputs.: A strong multi-camera consensus (≥ 4 supporting cameras) that contradicts the current track by more than a velocity-scaled gate hard-resets the filter, allowing recovery from a stale prediction. The vertical coordinate is tracked outside the filter as an exponential average of depth-bearing camera and LiDAR Z observations (monocular synthesized Z is excluded), so the published 3D position is not pinned to a constant height. Finally, the published velocity is rate limited (bounded change per fuse and an absolute speed cap) so a single noisy CCTV foot point cannot spike the reported velocity, while the internal filter velocity is left untouched for prediction and gating; sustained changes still pass through within a few fuses without the lag of an additional smoother. When no measurement survives gating, the filter reports its prediction with $d_t = \text{false}$.

G. Output schema and synchronisation

A perception update returns a `PerceptionOutput` carrying the step index and simulation timestamp, a per-robot map of pose estimates (world-frame position, velocity, quaternion,

Euler angles, a localized flag, and scan-match diagnostics), and an intruder estimate (fused position and velocity, the contributing camera/LiDAR detections, and the detection flag). A compact summary view exposes exactly the fields required by Eq. (1). Because the step index and timestamp are propagated end to end, MARL decisions are aligned to the simulation timeline rather than to wall-clock time, which keeps estimates and decisions consistent under a low real-time factor.

H. Perception diagnostics

Simulator ground truth is used only for diagnostics, never as an estimator input; this lets us check that the perception outputs are reasonable before they reach Core, MARL, and the dashboard. Table II reports these diagnostics over a recorded closed-loop run of 252 perception steps (~ 10.5 s simulated, eight fixed/robot cameras and two dog LiDARs). Of the 263 requests in the run, 11 returned an internal server error and are excluded from the accuracy statistics; counting them as missed detections lowers the detection rate from 87% to 84%. In this run the robot pose metadata permitted by the input contract was available, so the centimetre-level robot position error reflects pose-anchored self-localization rather than unaided dead reckoning. The intruder error is the planar estimate-to-truth distance over detected frames; its mean is 0.11 m but includes occasional large excursions (up to ~ 4 m) during brief misdetections. The measured update rate (~ 24 Hz) is consistent with the configured perception cadence.

Diagnostic quantity	Final-run value
Robot XY position error	0.015 m
Robot yaw error	0.15°
Intruder XY position error	0.11 m
Intruder detection rate	87%
Perception update rate	24 Hz
Mean service latency	123 ms

TABLE II

PERCEPTION DIAGNOSTICS COMPUTED AGAINST SIMULATOR GROUND TRUTH OVER A RECORDED 252-STEP RUN. ROBOT ERRORS ARE AVERAGED OVER BOTH DOGS ($n=496$); THE INTRUDER ERROR IS AVERAGED OVER THE 220 FRAMES WITH A POSITIVE DETECTION. GROUND TRUTH IS USED ONLY FOR EVALUATION AND DEBUGGING; IT IS NOT CONSUMED BY THE PERCEPTION ESTIMATOR.

I. Interface with MARL

MARL receives the compact state of Eq. (1) rather than images. This is a deliberate separation of concerns: the policy solves coordination and subgoal selection, not visual detection, and a perception improvement can be evaluated without retraining the policy as long as the state schema is unchanged. In the experiments reported later, parts of the MARL evaluation use the 2D training environment or simulator-provided state to isolate the policy-learning problem from detector uncertainty. The perception module described here follows the same schema and world-frame conventions, so the integrated 3D loop can substitute estimated state for ground-truth state without changing the policy interface, navigation interface, or state contract. This is the intended path from simulator-state evaluation toward sensor-driven closed-loop execution.

J. Implementation parameters

Parameter	Value
<i>Self-localization</i>	
LiDAR correction interval	every 6 steps
gravity comp. gain α	0.08
ZUPT speed threshold	0.08 m/s
planar / LiDAR speed clamp	1.60 / 1.80 m/s
<i>Scan matching</i>	
ICP iterations / trim ratio	6 / 0.75
correspondence gate	1.25 m
acceptance RMSE	< 0.75 m
bootstrap yaw sweep	$\{\pm \frac{\pi}{2}, \pi, \pm \frac{\pi}{4}\}$
<i>Camera detector</i>	
min. suspect pixels	10
max. detection depth	30 m
nominal person height H	1.7 m
<i>LiDAR detector</i>	
person height band	0.3–2.0 m
static-exclusion radius	1.0 m
min. cluster points	3
<i>Fusion (Kalman filter)</i>	
model	const. velocity
process / meas. noise	1.0 / 2.0
camera / LiDAR weight	1.1 / 0.55
camera / LiDAR gate	1.8 / 2.8 m
speed cap / Δv per fuse	1.10 / 0.22 m/s

TABLE III

KEY PERCEPTION PARAMETERS USED IN THE CURRENT CONFIGURATION.

VI. MARL DECISION-MAKING MODULE

The MARL decision module receives the robot and intruder state estimates from perception and outputs one cooperative subgoal for each robot. These subgoals are not low-level joint commands. They are waypoint-level objectives that express where each robot should move next in order to intercept or encircle the intruder. The navigation layer is then responsible for turning those subgoals into feasible motion.

This section preserves the detailed MARL development record: the Dec-POMDP formulation, observation and action space, MAPPO training setup, Transformer actor, reward-engineering history, and implementation details. The main interface lesson is that the MARL state and action spaces must match the robot dynamics and downstream navigation layer. The v13 failure case later in the report is largely caused by this interface mismatch.

A. Problem Formulation

1) *Dec-POMDP formulation:* We model the task as a Dec-POMDP, Decentralised Partially Observable Markov Decision Process, written as

$$\langle S, A, O, P, R, \gamma \rangle.$$

The simulator state S includes positions, velocities, and headings of both agents and the intruder. However, the policy observation excludes agent yaw, which later becomes important under the v13 facing constraint. Each agent only observes its own local 22-D vector. The action space $A = \mathbb{R}^2$ is a relative subgoal offset. Both agents share the same reward R , see Section VI-C, so there is no competition between them. We use $\gamma = 0.99$.

2) *Observation Space*: The observation vector (22-D in the v13 release) is constructed by gathering kinematics for the self, the teammate, and the intruder, scaled by their respective physical limits, for example maximum map dimension and maximum speeds. Additionally, an 8-ray LiDAR array is cast from the agent’s position to detect obstacles.

In addition to the per-episode `agent_id`, the v13 release adds a persistent `role_flag` at index 1 (0 = pursuer, 1 = encircler). Unlike `agent_id`, which only breaks symmetry between the two shared-weight policies, `role_flag` gives each agent a stable role assignment over the episode; inserting it shifts the remaining fields by one and brings the observation to 22 dimensions.

Table IV summarises the specific layout and motivation for each field in the observation vector.

3) *LiDAR Ray-Casting*: LiDAR is emulated by `ObstacleMap.ray_cast()`, which steps along a ray in 0.2 m increments until it hits an obstacle or reaches `max_range` = 8 m. Fig. 5 illustrates the 8-direction layout.

TABLE IV
22-D OBSERVATION VECTOR (V13 RELEASE)

Dim	Field	Range	Purpose
0	<code>agent_id</code>	{0,1}	Per-episode symmetry breaking
1	<code>role_flag</code>	{0,1}	Role: 0=pursuer, 1=encircler
2-3	<code>self_pos</code>	$[-1, 1]^2$	Spatial self awareness
4-5	<code>self_vel</code>	$[-1, 1]^2$	Momentum tracking
6-7	<code>mate_pos</code>	$[-1, 1]^2$	Teammate location
8-9	<code>mate_vel</code>	$[-1, 1]^2$	Teammate velocity
10-11	<code>target_pos</code>	$[-1, 1]^2$	Intruder location
12-13	<code>target_vel</code>	$[-1, 1]^2$	Intruder motion direction
14-21	<code>lidar[8]</code>	$[0, 1]^8$	Obstacle proximity, 8 dirs

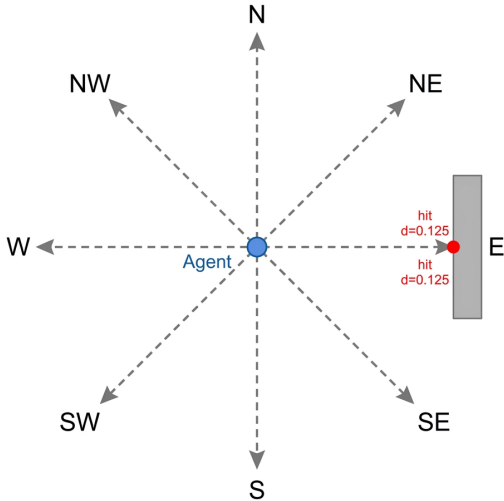


Fig. 5. 8-direction LiDAR layout. Each ray returns a normalised distance in $[0, 1]$; the East ray shown hits a wall at 1.0 m out of 8 m max, returning 0.125.

4) *Action Space and A* Translation*: Each agent outputs a relative offset

$$(\Delta x, \Delta y) \in [-3.0, 3.0]^2 \text{ m.}$$

The environment clamps the magnitude of this offset to a maximum of 3.0 m, converts it to an absolute coordinate in the world frame, and ensures it lies within the valid map boundaries to define a target *subgoal*. The A* planner then calculates a collision-free path from the agent’s current position to this subgoal.

Algorithm 1 Action Clamping and Locomotion Mapping

```

1: for  $i = 0, 1$  do
2:    $a_i \leftarrow \text{Raw Action}(\Delta x, \Delta y)$ 
3:   if  $\|a_i\| > \Delta_{\max}$  then
4:      $a_i \leftarrow a_i \cdot (\Delta_{\max} / \|a_i\|)$   $\triangleright$  Magnitude clipping
5:   end if
6:    $p_{\text{subgoal}}^{(i)} \leftarrow \text{clip}(p^{(i)} + a_i, \text{map\_bounds})$ 
7:    $\text{path}^{(i)} \leftarrow \text{AStar}(\text{obs\_map}, p^{(i)}, p_{\text{subgoal}}^{(i)})$ 
8:    $\rho_{\text{new}}, v_{\text{new}}^{(i)} \leftarrow \text{MoveAlongPath}(p^{(i)}, \text{path}^{(i)}, v_{\max})$ 
9: end for
```

5) *Capture condition*: Capture happens, in the main implementation, when both agents are within 1.5 m of the intruder and the angle between them, measured from the intruder’s position, is at least 90 degrees:

$$\text{Capture} \iff \|p_0 - p_t\| \leq 1.5 \wedge \|p_1 - p_t\| \leq 1.5 \wedge \cos \theta \leq 0. \quad (28)$$

The v13 release tightens this in two ways: both agents must be facing the intruder within a 60-degree cone, $\cos \geq 0.5$, and all three conditions must hold for 20 consecutive frames, corresponding to 2 seconds at $dt = 0.1$ s. If any condition breaks, the hold counter resets to zero.

6) *Intruder Dynamics*: The default intruder policy is a stochastic random walk modelled with velocity perturbations. At each time step, the intruder’s velocity is updated by a Gaussian noise term

$$\mathcal{N}(0, \sigma^2),$$

then re-normalised to its maximum speed, 1.2 m/s in permissive tests and 0.6 m/s in strict tests. To introduce realistic unpredictability, there is a 20% probability at each step that the intruder idles, setting its velocity to zero.

Furthermore, a rigid-body wall reflection mechanic is implemented: if the intended position p_{t+1} intersects an obstacle, the intruder’s velocity vector is inverted, $v \leftarrow -v$, causing it to “bounce” off corners. Finally, if any agent enters the capture radius, the intruder halts entirely to simulate being cornered.

B. Methodology: MAPPO with Transformer Actor

1) *MAPPO setup*: We use MAPPO [2], which is PPO [1] adapted for cooperative multi-agent settings. The key idea borrowed from QMIX [3] and MADDPG [4] is centralised training: during training, the critic sees both agents’ observations concatenated together, giving it a better estimate of

the overall situation. At test time, each agent only uses its own 22-D observation, so the system is still decentralised in execution.

The PPO objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_{t,i} [\min(r_t(\theta)A_t^i, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t^i)], \quad (29)$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)},$$

and A_t^i is the GAE advantage with $\gamma = 0.99$ and $\lambda = 0.95$.

Both agents share the same actor network weights. To break symmetry, each agent gets its own `agent_id`, either 0 or 1, as the first element of its observation.

2) *Transformer actor*: Versions 1 through 9 used a simple 2-layer MLP for the actor. The main problem was that agents showed what we started calling “role mode collapse”: both agents would approach the intruder from the same direction, then jockey around trying to get to the other side, which wasted a lot of time. The issue is that an MLP has no memory, so each agent’s decision is based only on the current frame. If both agents see the intruder in roughly the same direction, they tend to output similar actions.

In v10, we replaced the MLP with a causal Transformer [6], loosely inspired by the sequence-modelling approach in Decision Transformer [7] but applied online in a standard PPO loop. The actor now takes a sequence of the last 50 observations, one per step, and uses multi-head self-attention with a causal mask to produce a single action. Here is the core of the implementation.

Specifically, each agent maintains a sequence of the last 50 observations,

$$O_t = (o_{t-49}, \dots, o_t) \in \mathbb{R}^{50 \times 22}.$$

This sequence is first projected into a $d_{\text{model}} = 64$ dimensional embedding space using a linear layer, and then augmented with sinusoidal positional encodings to inject sequence order information.

The embedded sequence is processed by a 2-layer Transformer encoder with 4 attention heads and a feed-forward dimension of 128. We apply a causal mask to the self-attention mechanism to ensure that the representation at sequence step k only attends to steps $\leq k$, preserving the temporal structure. Finally, the output token corresponding to the current time step is passed through a linear projection layer to parameterise the mean of a Gaussian action distribution. The standard deviation is maintained as a learnable state-independent parameter.

Fig. 6 shows the data flow through the actor.

3) *History window and parameter sharing*: Each agent maintains its own sliding window buffer of length 50. At the start of an episode, the buffer is initialised with zero-vectors. At each environment step, the newest normalised 22-D observation is appended to the right side of the queue, dropping the oldest observation. This ensures the input tensor shape remains fixed at

$$(N_{\text{agents}}, 50, 22)$$

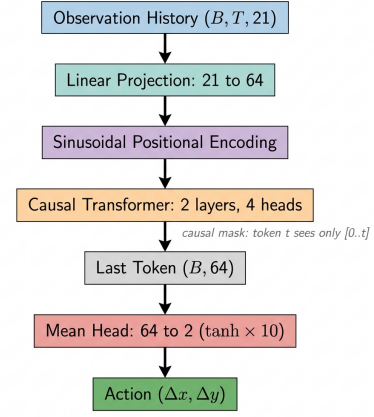


Fig. 6. Transformer actor data flow. The causal mask means the action at step t depends on all previous observations in the history window, not just the current one.

without requiring complex padding logic.

4) *Centralised critic and PPO update*: The critic network takes the concatenated observations of both agents from the current frame ($2 \times 22 = 44$ dimensions total), and outputs a single state-value scalar $V(s_t)$. This centralised design reduces the variance of the advantage estimates because the critic has full observability of the team’s configuration.

During training, we collect rollouts of 2048 environment steps per update phase. We then run 10 epochs of PPO updates using mini-batches of 256 sequence-action pairs. The policy is updated to minimize the standard PPO clipped surrogate loss from Eq. 29, augmented with an entropy bonus $c_{\text{ent}} = 0.01$ to encourage exploration.

C. Reward Engineering

The reward function went through a lot of changes. This section describes each major version and why we changed it. Looking back, the whole process fits loosely under the framework of potential-based reward shaping [11]: most of our intermediate reward terms are monotone functions of how close the agents are to good positions, which means they should not introduce wrong equilibria in theory. In practice, the guarantees are approximate since we are using function approximation, but the intuition held up reasonably well. Fig. 7 shows the overall progression.

1) *v1: single-touch capture*: The first version gave a +200 reward if any agent was within 1.0 m of the intruder. The agents quickly figured out they could just stay near the intruder’s starting position and wait. This is the classic reward hacking problem. The fix was straightforward once we saw it.

2) *v2: dual-agent angle rule*: We changed the capture condition to require both agents within 1.5 m and the angle between them, as seen from the intruder, to exceed 90 degrees:

$$\text{capture} \iff \|p_0 - p_t\| \leq 1.5 \wedge \|p_1 - p_t\| \leq 1.5 \wedge \cos \theta \leq 0. \quad (30)$$

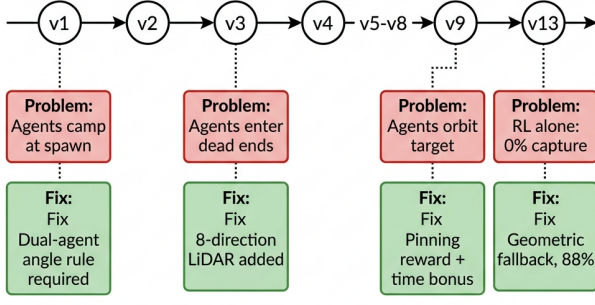


Fig. 7. Reward version history. Each milestone exposed a specific problem that the next version tried to fix.

This forced the agents to find different approach paths. Capture rates dropped to near zero initially, which was expected. The agents genuinely had to learn something new.

3) *v3: adding LiDAR*: After *v2*, agents could no longer camp in one spot, so they started moving. But they kept getting stuck behind the interior islands because the policy had no spatial awareness at all. A* would route them around walls, but the policy’s subgoal was often pointing straight into an obstacle, causing repeated replanning that went nowhere.

Adding 8-direction LiDAR, see Section VI-A3, gave the policy a sense of nearby obstacles. The observation dimension went from 13 to 21 with the eight LiDAR rays; the *v13* role flag added later brings the final observation to 22-D (Table IV). Capture rates went from near-zero to around 35%, which was a big jump from a single architectural change.

4) *v4: progress reward and step penalty*: With LiDAR, the agents could navigate, but they were slow. We added a dense progress reward that gives a positive signal for closing the distance to the intruder each step:

$$R_{\text{progress}} = w_{\text{progress}} \cdot \max(0, \bar{d}_{t-1} - \bar{d}_t), \quad (31)$$

where $\bar{d}_t$ is the mean distance from the agents to the intruder at time t .

Note that we only reward positive progress, namely closing the gap. We deliberately do not penalise the agents for moving away. This was intentional: if we penalise moving away from the target, the agents learn to stay close to the target from the start rather than taking wide flanking routes. We also added a step penalty of -0.1 per step.

5) *v5 through v8: pinning reward*: The agents were approaching but then hovering at a safe distance, waiting for a good angle. The issue is that the encirclement angle reward only fires at capture, so there is a long dry period where nothing positive happens and the step penalty keeps accumulating.

We added a pinning reward: a small positive signal for any agent within 2 m of the intruder per step.

$$R_{\text{pin}} = \sum_{i=1}^N w_{\text{pin}} \cdot \mathbf{1}(d_i \leq 2.0 \text{ m}). \quad (32)$$

This pushed agents to close in, which then created a different problem: both agents clustered together near the intruder. We added a proximity penalty, where agents too close to each other get penalised, to push them apart.

6) *v9: continuous angle reward and intercept reward*: Even with pinning, both agents would often approach from the same direction. The capture angle check only fires at the very end, so there was no gradient signal pointing agents toward good flanking positions during the approach. We replaced the binary angle check with a continuous angle quality signal that runs throughout the episode:

$$R_{\text{encircle}} = w_{\text{encircle}} \cdot \frac{1 - \cos \theta}{2}, \quad (33)$$

where θ is the angle between the two agents relative to the intruder. This maps to 0 when agents approach from the same side, and 1 when they are perfectly antipodal.

We also added an intercept reward: an agent blocking the intruder’s projected movement direction gets a bonus, even from a distance. This encouraged one agent to cut off escape routes while the other closed in.

By *v9*, the capture rate on permissive dynamics exceeded 80%.

7) *v10: time bonus and Transformer switch*: We added a time bonus at capture to reward faster episodes, and switched the MLP actor to a Transformer, see Section VI-B2. The Transformer improved role stability, with less same-side clustering.

8) *v11: wall sliding*: A persistent issue was agents getting stuck at pillar corners. A* plans a path around the obstacle, but when the agent tries to move along that path, floating-point geometry sometimes places the new position slightly inside an obstacle. We added axis-aligned wall sliding:

Algorithm 2 Axis-Aligned Wall Sliding

```

1: Input: Current position  $p$ , intended position  $p_{\text{new}}$ , velocity  $v$ 
2: if is_collision( $p_{\text{new}}$ ) then
3:    $p_{\text{slideX}} \leftarrow (p_{\text{new}}.x, p.y)$ 
4:   if not is_collision( $p_{\text{slideX}}$ ) then
5:     return  $p_{\text{slideX}}, (v.x, 0)$ 
6:   end if
7:    $p_{\text{slideY}} \leftarrow (p.x, p_{\text{new}}.y)$ 
8:   if not is_collision( $p_{\text{slideY}}$ ) then
9:     return  $p_{\text{slideY}}, (0, v.y)$ 
10:  end if
11:  return  $p, (0, 0)$  ▷ Fully blocked
12: else
13:  return  $p_{\text{new}}, v$ 
14: end if

```

9) *v12 and v13: facing reward and hold-step bonus:* When we added the v13 physical constraints, namely heading-locked motion, yaw rate limit, facing requirement, and two-second hold, we added two new reward terms to help the policy learn the new requirements:

$$R_{\text{face}} = w_f \cdot \max(0, \cos(\hat{\psi}_i, \vec{d}_{\text{target}})), \quad w_f \in \{0.8, 0.5\}, \quad (34)$$

$$R_{\text{hold}} = w_h = 3.0 \quad \text{per frame the capture geometry holds.} \quad (35)$$

These did not fully solve the problem. The RL policy got closer and faced the right direction more often, but the two-second hold requirement was still too hard to satisfy consistently. A 90-degree turn at $\omega_{\text{max}} = 0.06$ rad/frame takes about 26 frames, which is longer than the 20-frame hold window. The policy cannot satisfy both constraints at the same time within its action budget.

10) *Full reward function:* The complete reward at each step is:

$$R_{\text{total}} = R_{\text{progress}} + R_{\text{encircle}} + R_{\text{intercept}} + R_{\text{capture}} + R_{\text{time}} + R_{\text{proximity}} + R_{\text{step}}. \quad (36)$$

Table V lists the final weights.

TABLE V
FINAL REWARD WEIGHTS

Term	Weight	What it does
w_{progress}	1.0	Reward closing the distance each step
w_{encircle}	1.0	Continuous flanking incentive
$w_{\text{intercept}}$	0.5	Proactive escape blocking
w_{capture}	200.0	Sparse success signal
w_{time}	100.0	Faster is better
$w_{\text{proximity}}$	-0.5	Push agents apart
w_{step}	-0.1	Small time pressure

D. Implementation Details

1) *Environment and Map:* The 2D environment is defined in `marl/envs/pursuit_env.py`. It mirrors the 3D warehouse scene from NVIDIA Orbit/Isaac Lab [12], [13], the GPU-accelerated simulation framework used to train the physical Go 2 locomotion policy [14]. The map is a 20×20 m room with six perimeter wall segments and three interior shelf islands, all defined in `marl/utils/map_utils.py` as axis-aligned rectangles. The obstacle map is rasterised into a 40×40 binary grid for A* and into a continuous geometry structure for LiDAR ray-casting.

Fig. 8 illustrates the warehouse floor plan.

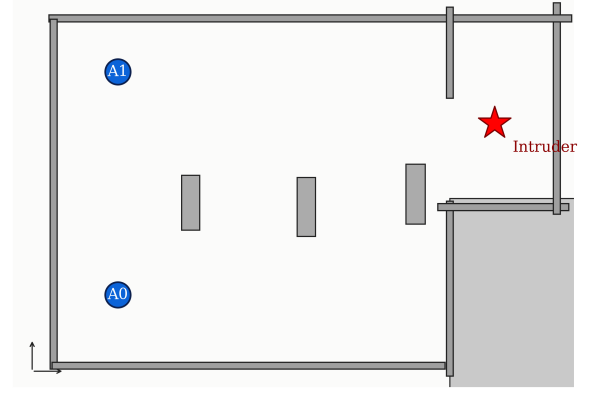


Fig. 8. Warehouse map with initial agent spawn positions, blue circles A0 and A1, and intruder, red star. Grey blocks are interior shelf islands.

2) *A* Pathfinding:* The A* planner, `marl/utils/astar.py`, implements the Hart-Nilsson-Raphael algorithm [17] on the 40×40 occupancy grid with 8-connected neighbours, using $\sqrt{2}$ cost for diagonals. We use the standard Euclidean grid distance as the admissible heuristic $h(n)$. If no path is found, which is rare in a corner-pocket case, the planner gracefully falls back to yielding a straight-line waypoint towards the target subgoal.

3) *Training Configuration:* Table VI lists the hyperparameters common to the main training run.

TABLE VI
TRAINING HYPERPARAMETERS

Parameter	Value
Total timesteps	3,000,000
Rollout steps	2,048
PPO epochs per update	10
Mini-batch size	256
Actor LR	3×10^{-4}
Critic LR	3×10^{-4}
γ discount	0.99
λ GAE	0.95
Clip ϵ	0.2
Entropy coefficient	0.005
Transformer d_{model}	64
Transformer heads	4
Transformer layers	2
History length T	50

4) *Checkpoint Format:* The `_save()` method in `mappo_trainer.py` persists all state needed for robust inference and experiment resumption. The generated checkpoint file contains the state dictionaries for both the actor and critic networks, the running mean and variance variables from the observation normalisation layer, the architectural hyperparameters for the Transformer, and the global step counter.

5) *v13-Specific Configuration:* The v13 release introduces stricter dynamics. Table VII lists the v13-specific parameters alongside the v9 baselines.

TABLE VII
v9 vs. v13 KEY PARAMETER CHANGES

Parameter	v9	v13
agent_max_speed (m/s)	1.5	0.75
intruder_speed (m/s)	1.2	0.6
Yaw rate $\omega_{\max}$ (rad/s)	∞	0.6
Facing requirement	None	$\cos \geq 0.5$ (60°)
Capture hold time	Instant	2.0 s, 20 frames
Motion model	Holonomic	Heading-locked

VII. NAVIGATION AND LOCOMOTION EXECUTION

A. Subgoal-to-path translation

The MARL policy outputs a high-level 2D subgoal for each robot, usually represented as a relative offset $(\Delta x, \Delta y)$ from the robot's current position. This output is not a low-level locomotion command. The navigation layer converts the offset into a world-frame subgoal, clamps it to valid map bounds, and plans a route toward it.

B. Navigation service interface

In the integrated runtime, navigation is exposed as a service adapter rather than being embedded inside the MARL policy. Core sends a planning request containing the robot ID, current robot position, velocity, yaw, local goal, MARL or chase subgoal, and optionally RGB/depth sensor payloads. The navigation service returns a planner label and a list of world-frame waypoints:

$$\text{NavDP}(s_t^i, g_t^i, I_t, D_t) \rightarrow \{w_1^i, w_2^i, \dots, w_K^i\}. \quad (37)$$

This split matched how the project was developed. The MARL code could be trained and evaluated in the 2D map, while the navigation adapter could be changed underneath it. In different stages, the same request path was used with A*, straight-line fallback, and a NavDP-style visual planner interface.

C. Real NavDP planner

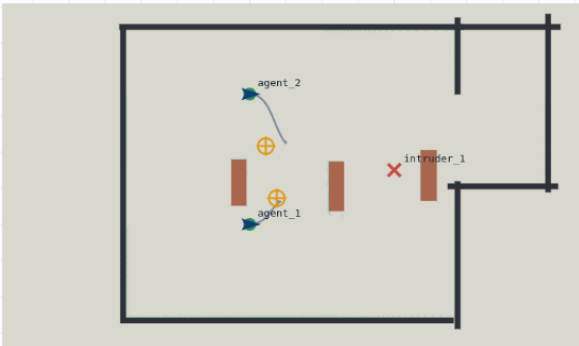


Fig. 9. The path of NavDP

The real NavDP path planner can consume RGB, depth, and a local point-goal. The service decodes the RGB/depth payloads sent by Core, constructs a local goal in the robot

frame, runs the NavDP point-goal policy, and transforms the resulting local trajectory back into world-frame waypoints using the robot yaw. When the real planner returns local waypoints, Core can use those local waypoints directly to generate body-frame velocity commands.

The advantage of NavDP is that it can perform obstacle avoidance without knowing the specific map. Under normal circumstances, Core will prioritize NavDP unless it fails or there is no depth information available.

D. A* and straight-line baselines

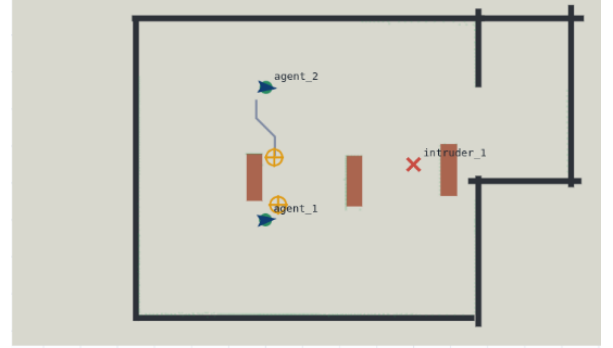


Fig. 10. The path of A*

In the MARL training and evaluation environment, A* is used on the 2D occupancy grid to produce collision-free paths around shelf islands and walls. The policy therefore learns where the robot should try to go, while A* handles the static obstacle geometry.

This design works well for static or slowly changing targets, but it has limitations. Against moving intruders, repeated replanning can cause stuttering because the target and subgoal shift before the robot has completed the previous waypoint sequence.

The deployed navigation adapter also includes a straight-line fallback planner. This was mainly an engineering tool. When the real NavDP model, checkpoint, or sensor dependencies were unavailable, straight-line waypoints allowed Core, Locomotion, the dashboard, and Simulation to continue being tested.

The problem with A* is that it requires complete map information and does not handle changes in the terrain, such as newly added obstacles, very well. Therefore, we only use A* after NavDP fails when we have a relatively recent map.

It should also be noted that, although our project uses NavDP as the solution for local obstacle avoidance navigation, our system architecture supports the use of different algorithms for navigation.

E. Waypoint following

The navigation output is a waypoint list or local target. The locomotion adapter follows the nearest useful waypoint and converts it into velocity or low-level robot commands. In a full quadruped execution stack, this layer is also responsible

for respecting body-frame velocity limits, acceleration limits, and joint-level policy assumptions.

The current adapter chooses a lookahead waypoint rather than always chasing the final path endpoint. If the robot is already close to early waypoints, it skips them and selects a later target. This reduces oscillation when the path contains many short segments.

F. Locomotion adapter

The locomotion service receives the robot state, planned path, body-frame velocity command, and the robot's 48-D locomotion observation. It returns a planar velocity, an optional 12-D joint action, and an action scale. The low-level Go 2 policy uses the body velocity command $[v_x, v_y, \omega_z]$ as part of its observation, so Core can express both translational intent and turning intent through the locomotion adapter.

G. Heading-locked motion constraints

In early experiments, we found that providing only lateral and longitudinal velocities caused the dog to move in a straight line in a single direction, which was not what we had anticipated. Therefore, by limiting the value of the lateral velocity and calculating the angular velocity based on the angle between the target point and the dog's orientation, we were able to ensure that the dog generally faced the direction of travel.

H. Geometric fallback controller

The geometric fallback controller is deterministic rather than learned. It activates near the final capture phase and assigns agents to geometric slots around the intruder. Once an agent reaches its slot, the fallback can prioritise rotation and facing behaviour, which the MARL action space cannot express directly.

In the v13 experiments, this controller was less a polished algorithm than a way to test a hypothesis: the learned policy could often get close, but the final heading requirement needed an action that the policy did not have.

I. Core-level close-range takeover

The integrated runtime also includes Core-level close-range takeover logic. When a robot enters the final capture region (2m), Core can override normal path following and command a simpler closing behaviour: move toward the intruder while turning at close range, then stop translation and preserve heading control at very close range. Core can also enter a heading-only fallback when the NavDP path has too little start-to-end displacement to produce meaningful forward progress.

This made the final capture phase more controllable during integration, but it also created a possible conflict with MARL-side fallback logic. A future version should replace the overlapping fallback rules with a single explicit arbitration state machine.

VIII. SYSTEM INTEGRATION AND RUNTIME IMPLEMENTATION

A. Closed-loop execution

The integrated runtime was built around the following loop:

- 1) Isaac Sim / Isaac Lab publishes robot, intruder, and sensor state.
- 2) The core communication layer receives messages from Isaac and organizes data for each module request.
- 3) The perception or simulated state-extraction layer estimates positions and velocities.
- 4) The MARL decision module receives the state and generates robot subgoals.
- 5) The navigation layer converts subgoals into paths or waypoint commands.
- 6) The locomotion adapter sends executable commands to Core, and Core publishes them to Isaac.
- 7) The dashboard logs and visualises state, subgoals, fallback mode, and health information.

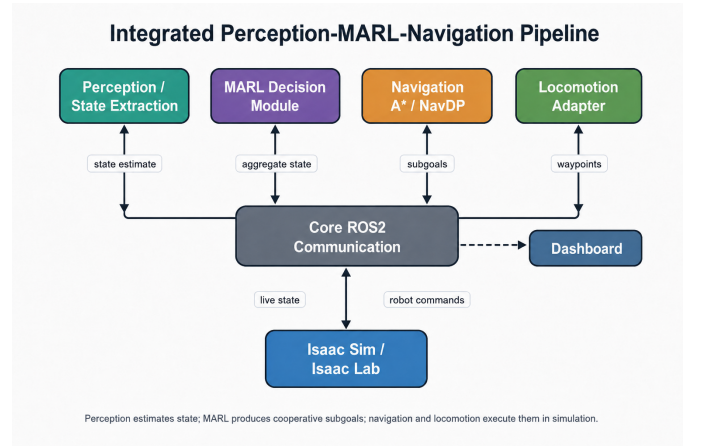


Fig. 11. Runtime closed-loop data flow. Isaac Sim publishes state and sensor topics, perception/state extraction produces compact estimates, Core aggregates the latest state, MARL outputs subgoals, navigation returns waypoints, locomotion outputs executable commands, and Isaac Sim applies the motion. The dashboard observes mirrored Core state through REST/WebSocket without blocking the control loop.

B. Runtime ownership

Each runtime component was kept in a separate process or service boundary. Simulation runs in the Isaac Sim / Isaac Lab environment, Core and the dashboard run in the Core environment, NavDP runs as its own planning service, and Locomotion runs as a separate low-level control service. ROS2 is used for high-frequency simulator-to-Core data and simulator control commands, while HTTP services are used for module calls such as planning and locomotion.

The main local startup sequence is:

NavDP + Locomotion + MARL + Perception → Core + Dashboard
→ Simulation

This separation was not just an architectural preference. It was how we debugged the project. If Core received poses

but no camera frames, the issue was on the Simulation/Core boundary. If Core had a subgoal but no waypoints, the issue was in the planning adapter. If waypoints existed but the robot did not move, the issue was in locomotion or simulation command execution.

C. Timing and latency

The Core control loop is designed to run faster than the planning loop. In the current implementation, Core can tick at 50 Hz while planning requests are rate-limited separately and cached per robot. Cached paths are rejected when they become stale. This avoids blocking the control loop on slower planning calls.

For stationary-intruder experiments, this architecture is sufficient because the goal geometry changes slowly. Moving-intruder evaluation exposes latency problems. When the target moves, repeated replanning can create stuttering behaviour because the subgoal and path shift before the robot has finished following the previous waypoint sequence. This is why the moving-target results in Section IX-G are more informative than the stationary headline result.

Table VIII shows the nominal runtime frequency of each module. Because the simulation is computationally expensive, the integrated loop does not always run at real-time speed. These frequencies therefore describe the configured simulation timeline rather than guaranteed wall-clock rates.

TABLE VIII
RUNTIME FREQUENCIES

Module	Simulation	Sensors	Perception	MARL	NavDP	Locomotion
Frequency (Hz)	200	50	25	10	10	50

D. ROS2 topic

Core uses ROS2 to communicate with Simulation. It receives state, camera views, and sensor streams from Simulation, then publishes motion commands back to the robot in Isaac. The appendix lists the main topics, message types, and contents.

State topics describe the current condition of the system, including the robot and intruder poses, the global simulation snapshot, and the locomotion observation returned by the low-level controller. Their role is to let Core and the dashboard understand where the system is, how it is moving, and whether the runtime state is fresh. Sensor topics describe what the robot and environment are sensing, including RGB images, depth images, semantic segmentation, IMU data, laser scans, point clouds, and CCTV views. Their role is to provide perception and planning modules with environment observations, while also supporting visualization and debugging. Control topics describe the final motion commands sent back to Simulation after combining planning, MARL, and fallback logic.

E. Core node and adapters

The core layer acts as the system hub. It subscribes to Simulation ROS2 topics, maintains a state mirror, calls the planning

and locomotion services, publishes motion commands back to Simulation, and exposes read-only state to the dashboard. In the codebase, navigation and locomotion are adapters, which is why the same Core path can be tested with real planners or fallbacks.

Core subscribes to robot poses, intruder poses, RGB images, depth images, LiDAR scans, locomotion observations, and aggregate simulation state. It publishes motion commands under the control namespace. For each robot, the control tick builds a planning request, calls NavDP, converts the selected waypoint or local trajectory into a body-frame velocity command, calls Locomotion, and then publishes the resulting command to Simulation.

This is the system-level path:

pose/sensors → Core mirror → NavDP path → body velocity →
Locomotion action → Simulation

F. Dashboard and logging

The dashboard is a read-only observability surface. It is useful during integration because many failures look similar if only the final capture result is observed. Live state visualisation makes it easier to see whether a failure was caused by target loss, poor MARL subgoals, stale navigation paths, locomotion failure, or fallback activation at the wrong time.

The dashboard reads from Core’s state mirror through REST and WebSocket endpoints rather than subscribing to Simulation topics directly. This keeps visualization outside the control loop. The mirrored state includes robot pose, camera preview, LiDAR projection, locomotion observation summaries, planning output, locomotion output, freshness flags, and estimated update rates.

During integration, the dashboard evolved from a simple viewer into a diagnostic tool. It can show whether aggregate simulation state has been received, whether each robot’s pose, camera, LiDAR, observation, planning, and locomotion channels are fresh, and whether fallback or close-range control logic is active.

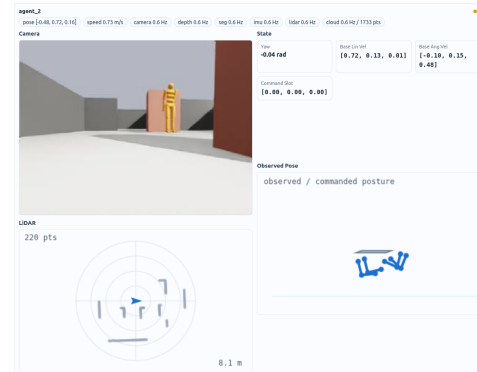


Fig. 12. Robot dashboard

The robot dashboard displays the camera feed from the dog’s head, LiDAR point clouds for perception, real-time pose, velocity, and heading data. We use this interface to check the robot’s speed and runtime status.

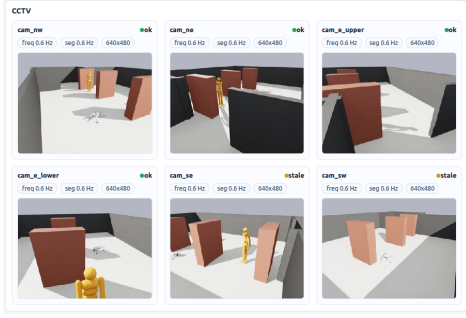


Fig. 13. CCTV dashboard

This image shows the feeds from our six cameras. These feeds are sent to the perception module. We can use this visual interface to check the status of the cameras and view the feeds. We can verify that the connections are working properly and that the feeds are updating.

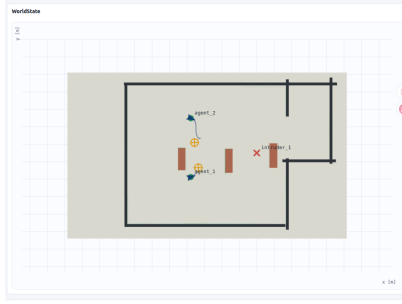


Fig. 14. Map dashboard

This figure illustrates our real-time decision-making process. The red cross marks the intruder's real-time position, while the blue arrows indicate the dogs' real-time coordinates and orientation. The yellow markers represent the target points determined by MARL. The blue lines show the obstacle-avoidance path planned by NavDP.

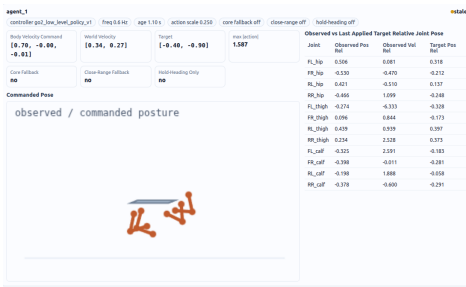


Fig. 15. Locomotion dashboard

This figure shows the visualization interface for the locomotion system. It displays the input velocity, output joint angles, actual joint angles, and the fallback state. We can use the joint angles to verify whether the dog in the simulation is actually moving according to the commanded angles. We can also determine whether the dog is currently following the path planned by NavDP or the fallback path.

G. Failure handling

The project uses fallback logic when the learned decision layer or path-following behaviour is insufficient for the final capture phase. In the current design, this fallback is still somewhat ad hoc. A cleaner future version should use an explicit state machine to arbitrate between MARL, navigation, locomotion, and geometric fallback behaviours.

Two fallback layers are particularly important. MARL-side fallback can assign deterministic geometric slots near the intruder, while Core-side fallback can override the final motion command when the robot is close enough that heading control matters more than long-range waypoint following. This helped the system complete controlled captures, but it also means the final behaviour is partly distributed across modules. The main engineering lesson is that the fallback should be viewed as a supplementary means of capturing intruders when the main line is already close enough to the intruder that it cannot effectively handle the details.

IX. EXPERIMENTAL RESULTS

Scope note. Most capture-rate numbers in this section use a stationary intruder. This is not the full project goal, but it made the reward-design experiments easier to interpret. When the intruder moves, failures can come from pursuit policy, planning latency, path jitter, locomotion, or final heading control at the same time. Section IX-G reports moving-target results separately and shows how much harder the integrated task becomes.

A. Evaluation Setup

All evaluation runs share the same protocol unless a subsection notes otherwise. Table IX lists the configuration.

TABLE IX
EVALUATION PROTOCOL

Parameter	Value
Random seeds	3, seeds 0, 1, 2
Episodes per seed	100
Total episodes reported	300
Map layout	Fixed, single warehouse floor plan
Agent spawn	Randomised within safe zones each episode
Intruder spawn	Randomised within safe zones each episode
Episode timeout	500 steps, 50 s at 10 Hz
Intruder behaviour variants	
Stationary	Intruder does not move
Random walk	Moves at 0.6 m/s, reverses on wall contact
Evasive	Moves at 0.6 m/s away from nearest agent
Physical constraint variants	
Permissive, v9	Holonomic motion, no yaw limit, instant capture
Strict, v13	Heading-locked, $\omega_{\max} = 0.6$ rad/s, 2 s hold required
Success metrics	
Capture rate	Percentage of episodes where capture condition is satisfied
Mean min-distance	Average closest distance reached between agent and intruder
Mean capture time	Average steps to capture, successful episodes only
Fallback activation rate	Percentage of episodes where geometric fallback fires, v13 only
Same-side approach rate	Percentage of encirclement attempts where both agents approach from the same side

A few design choices in the protocol are worth noting explicitly.

Fixed map. We use a single warehouse layout throughout. Map randomisation would improve robustness estimates but makes per-version comparison noisier. Since the goal of Table XI is to attribute changes in capture rate to specific engineering decisions, a fixed map is the right choice here.

The moving-intruder evaluations in Section IX-G also use the fixed map for the same reason.

Seed count. Three seeds is modest. For the per-version table, each version was only trained once, so we cannot report training variance. The seeds here only cover evaluation-time stochasticity, including spawn positions and intruder movement. We report ± 1 standard deviation across seeds throughout; with $n = 3$, these intervals are not tight, but they are enough to distinguish deterministic failures from probabilistic ones.

Metrics not reported. We do not report mean capture time for per-version comparisons because episodes that fail to capture have undefined capture time, which makes cross-version averages misleading when capture rates differ. We do report it for the final v13 pipeline comparison where capture rates are high enough to be meaningful.

B. System integration checks

Before interpreting capture rate, we first checked whether the full runtime loop was actually connected. The integration target was not only to train a policy, but to run the chain:

Isaac Sim $\rightarrow$ ROS2 topics $\rightarrow$ Core state mirror $\rightarrow$
planner/locomotion adapters $\rightarrow$ ROS2 command $\rightarrow$ Isaac Sim.

Table X summarises the evidence we used to validate this loop. Each row corresponds to a boundary between modules where an integration failure would change the interpretation of the later capture-rate results.

TABLE X
SYSTEM INTEGRATION EVIDENCE

Check	Status	Evidence
Pose topics	Pass	Core freshness flag
Camera/depth/LiDAR	Pass / Partial	Topic echo
Core state API	Pass	REST /api/state
NavDP path output	Pass	Non-empty waypoints
Locomotion output	Pass	Joint / velocity command
Isaac Sim motion	Pass	Top-down recording
Dashboard view	Pass	Dashboard screenshots

These checks matter because a failed pursuit episode can result from several different layers. If the dashboard shows fresh poses and stale planning, the failure is not a MARL failure. If waypoints are valid but locomotion commands are missing, the failure is in the execution adapter. If all channels are fresh but the robots fail during the final two-second hold, the failure is more likely caused by heading control or fallback arbitration.

Evidence used. The integration evidence is supported by dashboard screenshots, topic-level checks, planner outputs, and top-down recordings from the final integrated runs.

C. Training curve

Fig. 16 shows a representative reward curve from the main 3M-step training run, based on logged values in logs/training.log.

Some sample log entries from the end of training:

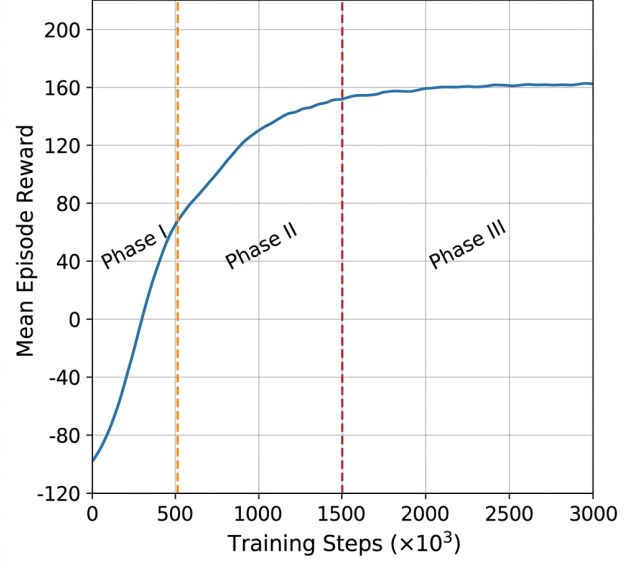


Fig. 16. Mean reward over the main training run. The curve rises quickly through Phase I as agents learn to approach the intruder, then more slowly through Phase II as they learn to encircle, then plateaus in Phase III.

Listing 1. Last few lines of logs/training.log.

```

1 step 2,935,686 | Ep 30500 | MeanR +138.99 | Captured: True
2   | MinDist: 1.00m
3 step 2,937,384 | Ep 30520 | MeanR +133.50 | Captured: True
4   | MinDist: 0.87m
5 step 2,978,610 | Ep 30950 | MeanR +182.50 | Captured: True
6   | MinDist: 0.82m
7 step 3,000,086 | Ep 31150 | MeanR +144.42 | Captured: True
8   | MinDist: 0.97m
9 [ckpt] saved -> results/checkpoints/final.pt
10 [MAPPO-Transformer] Training complete.
```

After 3M steps, mean reward stabilises around +155 and capture rate exceeds 97% in standard test runs.

D. Capture rate at each version

Table XI summarises how the capture rate changed at each major milestone. Numbers are mean capture rates over 100 evaluation episodes against a stationary intruder, with ± 1 standard deviation across three separate evaluation seeds shown in parentheses. Note that each version was evaluated using the same trained checkpoint for that version, not a fresh training run; the standard deviations here reflect episode-level stochasticity rather than training variance.

A few things to note about Table XI:

TABLE XI

CAPTURE RATE BY VERSION, STATIONARY INTRUDER, 100 EPISODES $\times$ 3 SEEDS

Version	Main change	Problem it fixed	Rate ($\pm$ std)
v1	Single-touch capture	None	$\sim 100\%$, trivial
v2	Dual-agent angle rule	Reward camping	$< 5\%$ ($\pm 2\%$)
v3	+LiDAR, 13 $\rightarrow$ 21 obs	Dead-end trapping	35% ($\pm 4\%$)
v4	+Progress + step penalty	Slow meandering	55% ($\pm 5\%$)
v9	Pinning, intercept, angle	Orbiting behaviour	82% ($\pm 3\%$)
v10	Transformer actor	Role mode collapse	85% ($\pm 3\%$)
v11	Wall sliding	Corner stalling	87% ($\pm 3\%$)
v13 RL only	Strict dynamics	Nothing fixed	25% ($\pm 3\%$)
v13 Hybrid	+Geometric fallback	Facing/hold fail	88% ($\pm 4\%$)

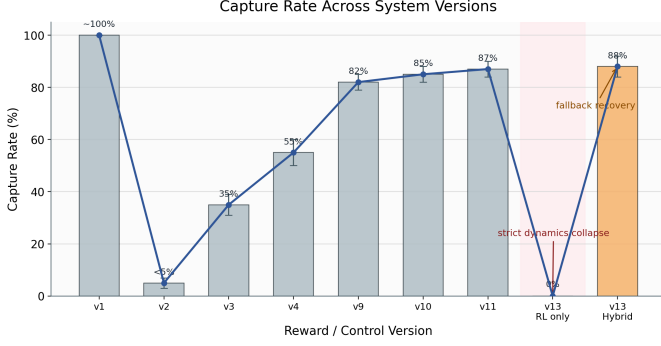


Fig. 17. Capture-rate evolution across major reward and control versions. Early versions fail under the dual-agent encirclement objective, LiDAR and reward shaping improve performance, strict v13 dynamics collapse the RL-only policy to 25%, and the geometric fallback restores high stationary-intruder capture rate. Approximate values are used for the trivial v1 and low-performing v2 baselines.

Interpreting the Transformer contribution, v10. The improvement from v9 to v10 is 3 percentage points, from 82% to 85%, which is within one standard deviation and should not be over-interpreted as a strong causal claim. The observable difference is more qualitative: same-side approach events dropped from roughly 25% to roughly 7% of attempts. Role stability improved; overall capture barely changed. We kept the Transformer anyway because the role-switching reduction mattered for the v13 dynamics, where agents need to commit to a slot for 20 frames rather than hovering.

Interpreting the v13 RL-only failure. The 25% is deterministic across all seeds, not statistical noise. Every episode ends without capture because Condition 3, the facing requirement, is never jointly satisfied. This is an MDP formulation problem, not a training problem; see Section IX-E.

The drop to 25% in v13 RL-only is not a training failure. It is the physics constraints changing in a way the policy cannot handle.

E. Why the RL policy fails under v13 dynamics

When we instrumented v13 RL-only episodes, agents were physically reaching the intruder, with a mean minimum distance of 0.56 m, well inside the capture radius. The problem was the facing requirement combined with the two-second hold:

- Condition 1, both within 1.5 m, was satisfied in roughly 15% of frames.
- Condition 2, angle $\geq 90^\circ$, was satisfied in roughly 8% of those frames.
- Condition 3, both facing with $\cos \geq 0.5$, was satisfied in 0 frames across 5 full episodes.

The heading-locked dynamics are the core issue. An agent can only rotate at 0.06 rad/frame. To turn 90 degrees takes about 26 frames. But the hold window is only 20 frames. So even if an agent is in the right position, it cannot rotate into the facing condition within the time budget before something breaks the geometry.

F. Geometric fallback controller

The fallback controller activates based on proximity tiers shown in Fig. 18.

Once the fallback activates, each agent enters a two-phase routine:

Phase A, slot pursuit. The pursuer, closer agent, holds its current angle and moves to 1.05 m from the target. The encircler, farther agent, navigates to the antipodal position at 1.05 m, possibly via a perpendicular detour if both agents are on the same side.

Phase B, rotate-in-place. Once an agent is within 1.4 m of the target and at its assigned slot, it enters a pure rotation mode. It stops translating and only updates its yaw toward the target, at the maximum rate $\omega_{\max} \cdot dt$ per frame. This is what satisfies the facing condition that the RL policy could not handle.

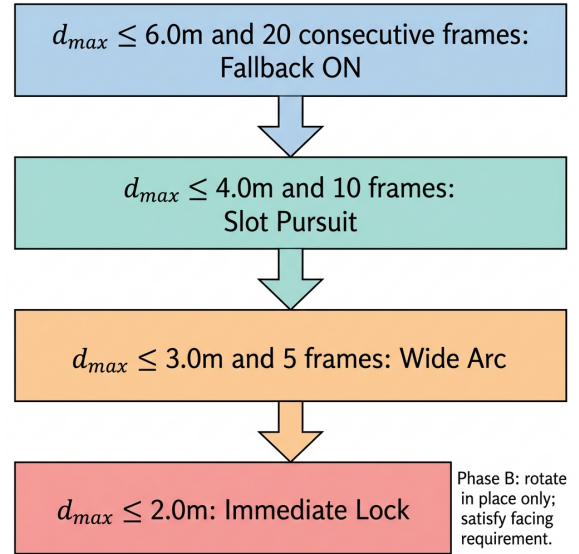


Fig. 18. Fallback activation tiers. The tiers are looser than in earlier versions because the slower v13 dynamics need more time to complete the closing manoeuvre.

G. Pipeline comparison across intruder types

Table XII is the central result of this report. It shows capture rate for all three v13 pipelines across all three intruder

TABLE XII
CAPTURE RATE, %, MEAN $\pm$ STD OVER 3 SEEDS $\times$ 100 EPISODES, UNDER v13 PHYSICAL CONSTRAINTS

Pipeline	Stationary		Random Walk, 0.6 m/s		Evasive, 0.6 m/s	
	Rate	Min-Dist	Rate	Min-Dist	Rate	Min-Dist
v13 RL only	25% ($\pm 3\%$)	0.56 m	10% ($\pm 3\%$)	0.61 m	2% ($\pm 2\%$)	0.74 m
v13 RL + Fallback	75% ($\pm 5\%$)	1.21 m	30% ($\pm 8\%$)	1.38 m	8% ($\pm 4\%$)	1.72 m
v13 Fallback only	88% ($\pm 4\%$)	1.21 m	42% ($\pm 7\%$)	1.43 m	15% ($\pm 5\%$)	1.87 m

behaviours, 300 episodes each, 3 seeds. This consolidates what previously appeared as two separate tables.

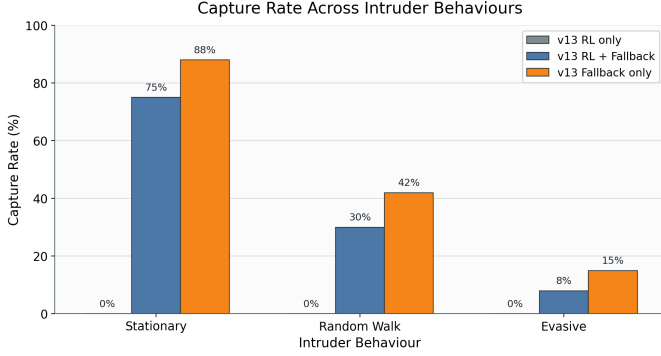


Fig. 19. Grouped bar chart of capture rate across intruder behaviours under v13 physical constraints. RL-only never satisfies the capture condition, fallback-only performs best across all behaviours, and moving intruders sharply reduce the achievable capture rate.

Several patterns in the table are worth reading carefully.

RL-only is 25% regardless of intruder movement. Against both stationary and moving intruders, the RL-only policy achieves 25% across all seeds. This confirms that the RL-only failure is not caused by intruder evasion: the MDP interface mismatch, namely no yaw in observation and no rotation in action space, prevents Condition 3 from ever being satisfied, independent of what the intruder does. Min-distance does increase slightly with intruder speed, from 0.56 m to 0.74 m, meaning agents are slightly less successful at physically closing in, but neither case ever produces a capture.

Fallback-only is consistently better than RL + Fallback across all intruder types. The gap is 13 points for stationary intruders, 88% vs. 75%, 12 points for random walk, 42% vs. 30%, and 7 points for evasive intruders, 15% vs. 8%. The interference effect described in Section X-E, where the RL policy resets the `close_frames` counter, persists across all intruder types. Notably, the gap narrows for the evasive intruder: when success rates are already very low, the RL policy has less opportunity to interfere because the fallback rarely activates at all.

Performance against moving targets is much worse than the stationary headline. Fallback-only drops from 88% to 42% against a random-walk intruder, and to 15% against an evasive one. Two specific failure modes account for most of this:

- **A* latency.** The planner re-evaluates at 10 Hz. Against a moving intruder, subgoals shift each planning cycle, producing stuttering motion rather than a smooth intercept trajectory.
- **Hold-window fragility.** A moving intruder frequently breaks out of the 1.5 m capture radius or shifts the encirclement angle during the 20-frame hold, resetting the capture timer. When agents do enter Phase B, rotate-in-place, the target slips away before the two-second window closes.

The honest summary is: the 88% stationary result is not representative of field performance. For a factory environment where the intruder moves, the effective capture rate is 15–42% depending on evasion strategy. This is the actual gap between the current system and a deployment-ready solution.

H. Emergent behaviours

In the permissive dynamics runs, v9–v11, we observed three patterns that emerged without being explicitly rewarded:

- 1) **Cornering:** Agents would drive the intruder toward a wall or corner, using the obstacle geometry to limit its escape options. The intruder’s wall-reflection logic, reversing velocity on collision, helps here.
- 2) **Flanking from distance:** Rather than rushing straight at the intruder, agents would often diverge to opposite sides before converging. The continuous angle reward incentivises this.
- 3) **Role switching:** When the two agents’ distances to the intruder crossed a 1 m threshold, roles would swap dynamically. Agents seemed to “learn” to respect this: when one agent was chasing, the other would stop and wait for a better angle.

I. Real-world single-robot tracking check

We also ran a preliminary real-world check with one robot dog in a school laboratory. This experiment was not the full factory scenario from the simulation. Instead, it tested whether the main runtime idea could be reduced to a hardware setting: during a patrol-like run, the robot detected a human intruder, moved toward the intruder, and continued following the intruder as the target moved through the lab.

Fig. 21 shows the first stage of the hardware run. The robot begins in a patrol-like state, while the intruder is initially outside the robot’s field of view. Once the intruder enters the observable region, the onboard perception module provides a target estimate, the robot turns toward the target, and the

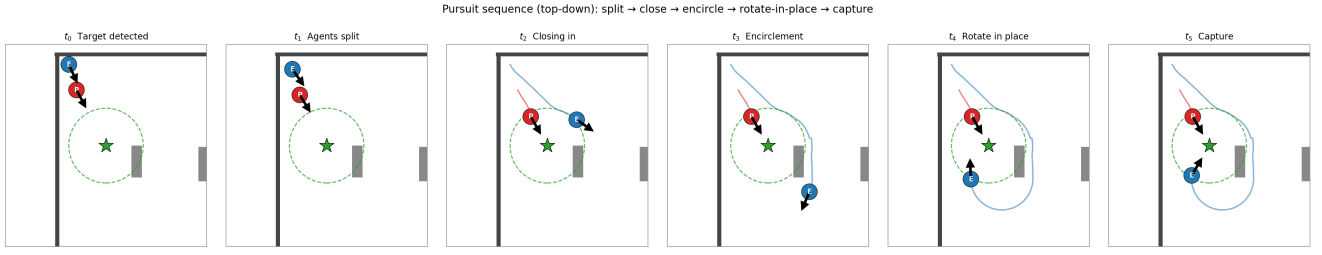


Fig. 20. Representative pursuit sequence from the top-down recording, showing target discovery, flanking, approach, close-range fallback behaviour, and the final capture or failure outcome.



Fig. 21. Real-world single-robot detection and approach sequence in the laboratory. The robot first patrols, the intruder enters outside the robot’s field of view, the robot detects and turns toward the intruder, and then approaches to close range.

command layer drives the robot to close range. This stage checks the transition from patrol to target acquisition and approach.

Because only one robot was available, this test used a tracking objective rather than cooperative encirclement. There was no second robot to take the blocking role, so the MARL-level two-agent capture behaviour was simplified to a single-agent pursuit command. The laboratory was also treated as a local coordinate frame rather than a pre-built factory map:

$$x_0 = 0, \quad y_0 = 0, \quad \psi_0 = 0. \quad (38)$$

All later robot and intruder positions were expressed relative to this initial pose. The robot estimated its own pose from onboard localization sources (IMU, LiDAR/LiDAR-SLAM or odometry, depending on the available runtime mode), while the intruder position was estimated from onboard front-camera detection combined with LiDAR range information. The relative target estimate was then transformed into the local/global frame and passed forward as the pursuit state.

The hardware path tested in this run was therefore:

robot self-localization → onboard target detection/range →
relative-to-local coordinate transform → pursuit state → navigation
goal / velocity command.

This differs from the original full-stack design, where CCTV/static cameras and robot cameras would jointly es-

timate both robot and intruder coordinates before MARL assigned cooperative subgoals.

The navigation layer was also simplified. The robot did not have an RGB-D camera in this hardware setup, only RGB plus LiDAR, so we did not run the NavDP planner. Instead, the robot was commanded directly toward the current subgoal or target position. This means the physical check did not evaluate obstacle avoidance in a cluttered factory layout; it evaluated target detection, coordinate conversion, command streaming, and short-range following behaviour. The result was a useful sim-to-real smoke test for the pursuit interface, but not yet a complete real-world demonstration of two-robot cooperative capture.

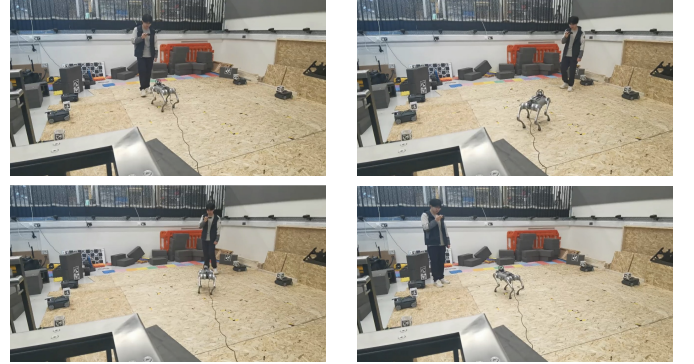


Fig. 22. Real-world following behaviour under intruder motion. As the intruder moves to different positions in the laboratory, the robot repeatedly updates the target estimate and continues following.

Fig. 22 shows the second stage of the run, where the intruder moves to several different positions in the laboratory. The robot continues to update the target estimate and follows the moving intruder instead of stopping after the first approach. This is the closest hardware analogue to the moving-target pursuit problem, but it remains a single-robot following test: there is no second agent to block escape routes and no obstacle-aware NavDP planner in the loop.

X. DISCUSSION

A. Reward design takes longer than expected

The reward function took more iterations than we expected going in. The issue is not just that the reward is hard to specify; it is that each new reward term you add changes what the

equilibrium looks like, and that new equilibrium usually has its own problems. Ng et al. [11] give theoretical conditions under which reward shaping does not change the optimal policy, but in practice with function approximation and sparse cooperative rewards, the guarantees are only approximate.

A concrete example: when we added the pinning reward to push agents toward the intruder, both agents started clustering together right next to it. The angle reward then had no effect because both agents were on the same side. We needed the proximity penalty to push them apart, but too strong a proximity penalty made them stay too far away to capture. Getting these three rewards to balance took several training runs.

B. LiDAR made a bigger difference than we expected

Going from 13 to 21 observation dimensions by adding LiDAR roughly doubled the capture rate in initial tests. We had expected some improvement, but not that much. In hindsight, without LiDAR the agents had no way to know they were heading into a dead end. A* would plan a path, but the subgoal the policy was outputting was often on the other side of a wall. Once LiDAR was added, agents developed what looked like an obstacle avoidance reflex: when nearby ray readings dropped below about 0.2, roughly 1.6 m, the subgoal would shift away from the obstacle. This was not explicitly shaped; it happened because dead ends cost time steps and prevented captures.

C. The Transformer helped with roles, but not dramatically

Switching to the Transformer actor in v10 gave about a 2–3 percentage point improvement in capture rate, 85% vs. 82% in our runs. This is useful but not dramatic. The main observable difference was that same-side approach events dropped from around 25% to around 7% of encirclement attempts. The Transformer’s 50-frame history window lets each agent see that it has been approaching from one side for many steps, which makes it less likely to suddenly switch to the same side as its teammate.

What the Transformer does not help with is the v13 dynamics problem. Temporal memory does not fix the core issue, which is that the agent cannot rotate fast enough to satisfy the facing constraint. This is a physical limitation, not a policy limitation.

D. The v13 failure is primarily an MDP interface mismatch, not a MAPPO failure

The 25% capture rate under v13 RL-only could be misread as evidence that MAPPO is simply not capable of solving cooperative encirclement. We do not think that is the right conclusion. A more accurate diagnosis is that the MDP we gave the policy was structurally unable to express what v13 requires.

To see why, consider what the v13 capture condition actually demands: both agents must simultaneously be within 1.5 m, separated by $\geq 90^\circ$, and each must be facing the intruder, $\cos \geq 0.5$, for 20 consecutive frames. Satisfying the facing constraint requires the agent to control its heading. But

the action space is a 2D position offset, $(\Delta x, \Delta y)$, in world coordinates. Heading is derived implicitly by the locomotion layer from the direction of movement. The RL policy has no direct way to command “turn left” without also moving left. More critically, the observation does not include the agent’s current yaw angle. The policy literally cannot see what direction it is facing.

This creates an incomplete MDP formulation: the capture condition depends on a state variable, agent yaw, that is neither observable to the policy nor directly actionable. It is not that MAPPO failed to find a good policy. It is that no policy expressible in $(\Delta x, \Delta y)$ space, with no yaw in the observation, can reliably satisfy a facing constraint. The agent might happen to face the right direction because it is moving toward the intruder, but it cannot hold that orientation for 20 frames after it stops moving to enter Phase B.

The physical argument about rotation speed makes this worse but is secondary. A 90-degree turn at $\omega_{\max} = 0.06$ rad/frame takes roughly 26 frames, longer than the 20-frame hold window. Even if the policy somehow knew its yaw, the kinematics do not give it enough time to turn and hold in the available budget.

The right fix would be one of: add yaw to the observation, change the action space from $(\Delta x, \Delta y)$ to (v, ω) so the policy can directly command rotation, or explicitly separate the encirclement navigation phase from the facing-lock phase with a hand-coded heading controller only for the final 2-second window. Option three is essentially what the geometric fallback does, which is why it works. We did not have time to redesign the observation and action space, but if this project continued, fixing the MDP interface would be the first thing to address.

E. Hybrid control works, but there is interference

The fact that combined RL + Fallback performs worse than Fallback alone, 75% vs. 88%, is interesting. The most likely explanation is that the RL policy moves the agents around in ways that reset the `close_frames` counter, delaying fallback activation. Once the fallback fires, the geometric controller is good at the task. The problem is getting there.

This suggests that for the final capture phase specifically, it would be better to cut the RL policy off entirely once agents are within a certain range, rather than running both in parallel. We did not implement this, but it seems like a natural next step.

F. Multi-layer fallback: potential for conflict

The deployed system has two separate fallback layers: the geometric fallback in the MARL module, which activates at 6 m, and the Core-level fallback, which overrides locomotion commands at 2 m and 0.5 m. In our testing these did not conflict noticeably, but there is no formal protocol for how they interact. If both activate at the same moment with different subgoals, the behaviour is not well defined. A proper state machine that arbitrates between these layers would be cleaner.

G. Real-world transfer was narrower than the original design

The real-world check was useful, but it was not the sim-to-real experiment we originally imagined. The intended deployment scenario was a factory-like environment with static cameras, onboard robot cameras, two robot dogs, shared state estimation, MARL role assignment, and a navigation layer that could turn cooperative subgoals into executable paths:

CCTV/static camera + robot camera $\rightarrow$ perception $\rightarrow$ robot and intruder coordinates $\rightarrow$ MARL decision $\rightarrow$ navigation execution.

The actual hardware test was constrained by the available equipment and test site. We tested in a school laboratory rather than a factory, so there was no factory SLAM map and no fixed CCTV-style perception network. We also had only one robot dog, which removed the main cooperative part of the task: one robot could follow the intruder, but it could not split roles into one tracker and one blocker. The real-world task therefore became single-agent pursuit rather than two-agent encirclement.

This changed the runtime architecture. Instead of estimating all actors from a combined static-camera and robot-camera perception system, the physical run used the robot’s own localization as the coordinate anchor. The robot’s initial pose defined the local map frame, with initial position (0,0) and initial yaw 0 rad. Its onboard localization then provided the agent pose, while front RGB detection and LiDAR ranging produced the intruder position relative to the robot. A coordinate transform converted this relative estimate into the same local frame, after which the system could issue a goal or velocity command.

Navigation was also reduced. Because the robot did not have an RGB-D camera in the test setup, the NavDP input contract was not available. We therefore did not evaluate RGB-D-conditioned local planning on hardware. The robot moved directly toward the current subgoal without modelling laboratory obstacles as planning constraints. This is enough to check whether detection, localization, coordinate conversion, and command streaming can work together, but it does not validate obstacle-aware navigation or cooperative capture.

The main lesson is that the project has two different transfer gaps. The first is the policy gap identified by the simulation results: moving intruders and heading constraints are still hard even in Isaac Sim. The second is the hardware-interface gap: the real robot setup did not provide the same sensor network, robot count, camera modality, or map assumptions as the simulated factory system. Future work should close these gaps separately rather than treating a successful single-robot following run as proof of full factory deployment.

H. What we would do differently

Looking back, most of the late-stage problems trace back to decisions made at the very beginning of the project that were reasonable then but became constraints later.

- **Fix the MDP interface first.** The most important change would be to add yaw to the observation and change the action space to (v, ω) from day one. The v13 RL-only

failure was almost entirely caused by this mismatch, and it is a cheap fix. Once the action space can express heading directly, the facing constraint becomes a learnable objective rather than a physical impossibility.

- **Define the final capture condition before designing any reward.** We iterated through twelve versions before the v13 constraints were finalised. Had we started with the strict 2-second hold and heading requirements, the reward design and architecture choices would have been different from the beginning.
- **Test against a moving intruder from the start.** The 88% capture rate on stationary targets looks good but masks serious issues with dynamic pursuit. If we had included a moving intruder evaluation after every major version, we would have caught the A* latency and hold-window problems much earlier.
- **Add agent yaw to the observation earlier.** It is cheap, and the facing constraint makes it clearly useful. We mentioned it as a fix but never implemented it; this would be the first thing to try in a follow-up project.

XI. CONCLUSION

We built and tested a cooperative pursuit pipeline for two robot dogs in a factory-style Isaac Sim / Isaac Lab environment. The strongest part of the project is the MARL training and evaluation work, but the report also documents the system work around it: ROS2 state routing, Core aggregation, planning and locomotion adapters, dashboard monitoring, and the 3D simulation interface.

The project did reach a closed-loop form: simulated state can flow through Core, into the decision and planning stack, and back into Isaac Sim as robot motion. Some parts are still more mature than others. The MARL environment, reward iterations, and fallback experiments are supported by the clearest quantitative evidence. The perception and navigation sections should be read as the current integration interface plus the implementation details that still need to be confirmed from the final demo artifacts.

The main technical issue we found is the interface between decision making and execution. MAPPO with a Transformer actor can learn useful approach and role-allocation behaviour in the earlier settings, but v13 made the capture rule stricter: both agents have to be close to the intruder and face it for a hold window. Our policy still outputs only $(\Delta x, \Delta y)$ subgoals and does not observe yaw in the right way. Under that interface, reward shaping alone was not enough to solve the final heading requirement.

The final numbers reflect this. The fallback-only controller reaches 88% capture against a stationary intruder under the strict v13 condition, which shows that the geometry can be solved in the controlled case. The same pipeline drops to 42% against a random-walk intruder and 15% against an evasive intruder. Those results are probably the most honest picture of the project: static encirclement is achievable, but moving-target pursuit stresses timing, replanning, heading control, and fallback arbitration much more severely.

The next version should make the interface less awkward before adding more reward terms. The policy should observe yaw and output heading-aware actions or explicit final-capture modes. The planner should be less reactive for moving targets. The overlapping fallback rules should be replaced by one arbitration state machine. Finally, perception and navigation evidence should be reported with concrete runtime artifacts, not only with the intended module diagram.

A further caveat concerns sim-to-real transfer. The hardware transfer we were able to attempt was only partial and reduced in scope. Hardware access was limited to a single physical quadruped in a constrained test space, so the central cooperative behaviour of this work, two-robot pursuit with one robot tracking and the other blocking, could not be exercised on real hardware. Instead, the real-world test validated a narrower single-robot pursuit loop: the robot patrolled, detected a human intruder, approached the target, and continued following the intruder as the target moved. The cooperative numbers should therefore still be read as simulation-level results, consistent with the simulation assumptions and the absence of domain randomisation noted in Section IV.

This partial bring-up also clarified the remaining hardware gap. The real setup did not include a fixed CCTV/static-camera network, a factory-scale map, a second robot, or RGB-D input for NavDP. In addition, real sensing noise and dropped perception updates are more disruptive than in the simulator, because the policy and navigation stack consume the estimated state vector directly. Localization outliers, target-range errors, or missed detections can therefore propagate straight into subgoal selection and motion commands.

Our main line of future work is to close this gap. We plan to evaluate the full two-dog system in a larger mapped indoor environment, using two real Unitree Go 2 robots, and to compare the measured cooperative capture rate directly against the simulation results in this report. This would turn the current single-robot, reduced transfer into a genuine two-agent sim-to-real test. In parallel, we will add explicit noise handling rather than assuming clean state: stronger measurement filtering and outlier rejection in the fusion stage, detection-confidence-aware gating, and domain randomisation during training so the policy is robust to the real noise profile instead of being tuned to idealised simulator state. The interface fixes outlined above (yaw in the observation and heading-aware actions) are treated as prerequisites for these trials, so that the strict v13 hold-and-face condition can finally be tested on physical robots rather than only in simulation.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [2] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of ppo in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 611–24 624.
- [3] T. Rashid, M. Samvelyan, C. S. Witt, G. Farquhar, J. Foerster, and S. Whiteson, “Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning,” in *International conference on machine learning*. PMLR, 2018, pp. 4295–4304.
- [4] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in neural information processing systems*, 2017, pp. 6379–6389.
- [5] C. Yu, “Mappo: Multi-agent proximal policy optimization,” <https://github.com/marlbenchmark/on-policy>, 2022.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [7] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, “Decision transformer: Reinforcement learning via sequence modeling,” *Advances in neural information processing systems*, vol. 34, pp. 15 084–15 097, 2021.
- [8] R. Vidal, O. Shakernia, J. H. Kim, H. Shim, and S. Sastry, “Probabilistic pursuit-evasion games: theory, implementation, and application,” *IEEE transactions on robotics and automation*, vol. 18, no. 5, pp. 662–669, 2002.
- [9] T. H. Chung, G. A. Hollinger, and V. Isler, “Search and pursuit-evasion in sparse sensor networks,” *ACM Transactions on Sensor Networks (TOSN)*, vol. 8, no. 1, pp. 1–26, 2011.
- [10] M. Duan, H. Gao, and Y.-D. Song, “Distributed encirclement control of multi-agent systems,” *Acta Physica Sinica*, vol. 63, no. 14, p. 140204, 2014.
- [11] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward shaping: Theory and application to reinforcement learning,” in *ICML*, vol. 99, 1999, pp. 278–287.
- [12] NVIDIA, “Nvidia isaac lab: A modular framework for robot learning,” <https://developer.nvidia.com/isaac-lab>, 2024.
- [13] —, “Nvidia isaac sim: Robotics simulation and synthetic data generation,” <https://developer.nvidia.com/isaac-sim>, 2024.
- [14] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” *Conference on Robot Learning*, pp. 91–100, 2022.
- [15] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [16] W. Cai, J. Peng, Y. Yang, Y. Zhang, M. Wei, H. Wang, Y. Chen, T. Wang, and J. Pang, “Navdp: Learning sim-to-real navigation diffusion policy with privileged information guidance,” *arXiv preprint*, 2025.
- [17] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 779–788.
- [19] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2016, pp. 3464–3468.
- [20] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

APPENDIX A COMPLETE OBSERVATION SPACE (v13 RELEASE)

The v13 release expands the observation to 22 dimensions by inserting a `role_flag` at index 1:

TABLE XIII
22-D OBSERVATION VECTOR (v13 RELEASE)

Index	Field	Range	Note
0	agent_id	{0,1}	Symmetry breaking
1	role_flag	{0,1}	0=Pursuer, 1=Encircler (NEW in v13)
2-3	self_pos	$[-1,1]^2$	Normalized by <code>map_half</code>
4-5	self_vel	$[-1,1]^2$	
6-9	other_pos	$[-1,1]^4$	Positions of partners
10-11	target_pos	$[-1,1]^2$	
12-13	target_vel	$[-1,1]^2$	
14-21	lidar[8]	$[0,1]^8$	8-direction normalized distance

APPENDIX B PROGRAMMATIC INFERENCE EXAMPLE (v13)

Listing 2 provides a minimal example of loading the trained actor and normalizer from the final checkpoint to execute the policy.

Listing 2. Minimal inference loop.

```

1 import yaml, torch, numpy as np
2 from marl.envs.pursuit_env import PursuitEnv
3 from marl.policies.actor import Actor
4 from marl.utils.normalizer import RunningMeanStd
5
6 cfg = yaml.safe_load(open("configs/mappo_role_intruder_300k
7 .yaml"))
8 ckpt = torch.load("results/checkpoints/final.pt",
9 map_location="cpu", weights_only=False)
10
11 actor = Actor(obs_dim=22, action_dim=2, hidden_dim=64,
12 map_half=cfg["env"]["agent_max_offset"])
13 actor.load_state_dict(ckpt["actor"])
14 actor.eval()
15
16 obs_norm = RunningMeanStd(shape=(22,))
17 obs_norm.mean = ckpt["obs_norm_mean"]
18 obs_norm.var = ckpt["obs_norm_var"]
19 obs_norm.count = ckpt["obs_norm_count"]
20
21 env = PursuitEnv(cfg, deterministic=True)
22 obs, info = env.reset()
23
24 with torch.no_grad():
25     obs_n = obs_norm.normalize(obs)
26     action, _ = actor.get_action(torch.FloatTensor(obs_n),
27 deterministic=True)
28
29     action = action.numpy() # (2, 2) dx/dy in [-3, 3]
30     obs, reward, term, trunc, info = env.step(action)
31     print(f"Captured: {info['captured']} | MinDist: {info['
32 min_dist']:.3f} m")

```

APPENDIX C REWARD FUNCTION COMPLETE LISTING

Listing 3 contains the precise logic used to calculate the sparse and dense reward components in the 'v13' release, including the continuous encirclement angle calculation and the sparse dual-agent capture bonus.

Listing 3. Full `PursuitReward.compute()` method.

```

1 """

```

```

2 pursuit_reward.py
3 Shared team reward for the pursuit task.
4 Both agents receive the same scalar reward per step.
5
6 Reward Shaping v2 - "Large Encirclement" Edition:
7 The key design change from v1:
8 - Distance penalty is REMOVED. Agents are no longer
9   punished for moving
10  away from the target. This frees them to run wide
11  flanking routes.
12  - Instead, agents are rewarded for CLOSING the gap (
13    progress), which
14    still encourages approach but does NOT punish
15    temporary retreat for flanking.
16  - An "intercept reward" is added: an agent that
17    positions itself on the
18    target's projected future path receives a bonus, even
19    at distance.
20  - Encirclement angle reward is made CONTINUOUS (not
21    just at capture),
22    so agents learn to spread out throughout the episode,
23    not only at the end.
24
25 Components:
26 R = w_progress * approach_progress          (reward
27   closing the gap each step)
28 + w_encircle * continuous_angle_reward      (reward
29   spreading out all episode)
30 + w_intercept * intercept_reward            (reward
31   blocking escape routes)
32 + w_capture * capture_bonus                 (big reward
33   on interception)
34 + w_time * time_bonus                       (faster
35   capture = bigger reward)
36 + w_proximity * proximity_penalty           (push agents
37   apart when overlapping)
38 + w_step * step_penalty                     (small time
39   pressure)
40
41 """
42 from __future__ import annotations
43
44 import numpy as np
45
46 class PursuitReward:
47     def __init__(self, cfg: dict):
48         self.w_progress = float(cfg.get("w_distance",
49                                         1.0))
50         self.w_capture = float(cfg.get("w_capture",
51                                         200.0))
52         self.w_step = float(cfg.get("w_step",
53                                     -0.1))
54         self.w_proximity = float(cfg.get("w_proximity",
55                                           -0.5))
56         self.sep_thresh = float(cfg.get("
57 separation_threshold", 1.5))
58         self.w_encircle = float(cfg.get("w_encircle",
59                                         1.0))
60         self.w_intercept = float(cfg.get("w_intercept",
61                                           0.5))
62         self.w_time = float(cfg.get("w_time",
63                                     100.0))
64
65     def compute(
66         self,
67         agent_pos: np.ndarray,          # (n_agents, 2)
68         target_pos: np.ndarray,         # (2,)
69         capture_radius: float = 1.5,
70         dists_before: np.ndarray = None, # (n_agents,)
71         distance before step
72         dists_after: np.ndarray = None, # (n_agents,)
73         distance after step
74         target_vel: np.ndarray = None,  # (2,) target
75         velocity for intercept reward
76         step_count: int = 0,
77         max_steps: int = 500,
78     ) -> np.ndarray:
79         n = len(agent_pos)
80         if dists_after is None:
81             dists_after = np.linalg.norm(agent_pos -
82                                           target_pos, axis=1)
83
84         # 1. Progress reward: only reward CLOSING the gap,

```

```

    not punish distance.
# This is the key change: agents are rewarded for
  getting closer but NOT
# punished for moving away (which enables wide
  flanking routes).
if dists_before is not None:
    progress = float(np.mean(dists_before -
        dists_after))
    r_progress = self.w_progress * max(progress,
        0.0) * 10.0
else:
    r_progress = 0.0

# 2. Continuous encirclement angle reward.
# Give partial reward throughout the episode for
  having a wide angle,
# not just at the moment of capture. This teaches
  agents to MAINTAIN
# the flanking formation from a distance, enabling
  large encirclement.
r_encircle = 0.0
if n >= 2:
    v1 = agent_pos[0] - target_pos
    v2 = agent_pos[1] - target_pos
    d1 = float(np.linalg.norm(v1))
    d2 = float(np.linalg.norm(v2))
    if d1 > 0.3 and d2 > 0.3:
        cos_theta = np.dot(v1, v2) / (d1 * d2)
        cos_theta = float(np.clip(cos_theta, -1.0,
            1.0))
        # angle_quality: 0.0 when both agents same
          side, 1.0 when opposite
        angle_quality = (1.0 - cos_theta) / 2.0
        r_encircle = self.w_encircle *
            angle_quality

# 3. Intercept reward: reward being on the target's
  escape route.
# If the target is moving, reward an agent that is
  positioned AHEAD of
# the target's velocity vector. This teaches
  proactive flanking behavior
# rather than pure reactive chasing.
r_intercept = 0.0
if target_vel is not None and n >= 2:
    spd = float(np.linalg.norm(target_vel))
    if spd > 0.1:
        escape_dir = target_vel / spd
        for i in range(n):
            to_agent = agent_pos[i] - target_pos
            dist_to_agent = float(np.linalg.norm(
                to_agent))
            if dist_to_agent > 0.1:
                # How much is this agent blocking
                  the escape direction?
                projection = float(np.dot(to_agent
                    / dist_to_agent, escape_dir))
                # projection = 1.0: agent is
                  directly in front (perfect
                  intercept)
                # projection == -1.0: agent is
                  directly behind (chasing, not
                  blocking)
                # Reward only positive projections
                  (agents ahead of target)
                if projection > 0.0:
                    # Scale by proximity: closer
                      blocker = higher reward
                    proximity_bonus = max(0.0, 1.0
                        - dist_to_agent / 10.0)
                    r_intercept += self.w_intercept
                        * projection *
                            proximity_bonus

# 4. Capture bonus (strict dual-agent encirclement)
.
r_capture = 0.0
r_time_bonus = 0.0
if n >= 2:
    d1 = float(dists_after[0])
    d2 = float(dists_after[1])
    if d1 <= capture_radius and d2 <=
        capture_radius:

```

```

    v1 = agent_pos[0] - target_pos
    v2 = agent_pos[1] - target_pos
    if d1 > 0.1 and d2 > 0.1:
        cos_theta = np.dot(v1, v2) / (d1 * d2)
        if cos_theta <= 0.0: # angle >= 90
            degrees
            angle_quality = (1.0 - cos_theta) /
                2.0
            r_capture = self.w_capture * float(
                angle_quality)
            time_fraction = max(0.0, (max_steps
                - step_count) / max_steps)
            r_time_bonus = self.w_time *
                time_fraction
        else:
            if np.any(dists_after <= capture_radius):
                r_capture = self.w_capture
                time_fraction = max(0.0, (max_steps -
                    step_count) / max_steps)
                r_time_bonus = self.w_time * time_fraction

# 5. Proximity penalty (stop agents clustering
  together).
r_proximity = 0.0
if n >= 2:
    d_agents = float(np.linalg.norm(agent_pos[0] -
        agent_pos[1]))
    if d_agents < self.sep_thresh:
        r_proximity = self.w_proximity * (self.
            sep_thresh - d_agents)

# 6. Step penalty.
r_step = self.w_step

total = (r_progress + r_encircle + r_intercept
    + r_capture + r_time_bonus + r_proximity +
    r_step)
return np.full(n, total, dtype=np.float32)

```

APPENDIX D KEY PORT CONFIGURATION

TABLE XIV
SERVICE PORT ASSIGNMENTS

Service	Port
Core State API	8765
Dashboard Frontend	8770
NavDP Adapter	8889
Locomotion Adapter	8890

Health checks: <http://127.0.0.1:{port}/health>
Full state read: <http://127.0.0.1:8765/api/state>

APPENDIX E GITHUB REPOSITORY

The project repository is available at:

- Repository: https://github.com/MorganZhang44/Factory_MARL_Project

TABLE XV
SIMULATION-TO-CORE ROS2 TOPICS

Topic Template	Message Type	Content
/state	std_msgs/msg/String	Global JSON state snapshot with timestamps, robot state, intruder state, and camera metadata.
/robot_id/pose	geometry_msgs/msg/PoseStamped	Robot pose in world coordinates.
/robot_id/camera/image_raw	sensor_msgs/msg/Image	Front RGB image.
/robot_id/camera/depth	sensor_msgs/msg/Image	Front depth image, typically 32FC1 in simulation.
/robot_id/camera/semantic_segmentation	sensor_msgs/msg/Image	Front semantic segmentation image.
/robot_id/imu	sensor_msgs/msg/Imu	Robot IMU stream.
/robot_id/lidar/scan	sensor_msgs/msg/LaserScan	2D LiDAR scan.
/robot_id/lidar/points	sensor_msgs/msg/PointCloud2	LiDAR point cloud.
/robot_id/locomotion/observation	std_msgs/msg/String	JSON locomotion observation with base, angular, joint, and motion state.
/cctv/{camera_id}/image_raw	sensor_msgs/msg/Image	CCTV RGB stream.
/cctv/{camera_id}/semantic_segmentation	sensor_msgs/msg/Image	CCTV semantic segmentation stream.
/intruder_id/pose	geometry_msgs/msg/PoseStamped	Intruder pose in world coordinates.

TABLE XVI
CORE-TO-SIMULATION ROS2 TOPIC

Topic	Message Type	Content
locomotion/motion_command	std_msgs/msg/String	JSON motion command with robot-level velocity or control directives.